

Innhold

1	Interpolasjon (MAT20x)	1
1.1	Interpolasjon i en dimensjon (MAT20x)	2
1.2	Lineær interpolasjon (MAT20x)	6
1.3	Polynomisk interpolasjon (MAT20x)	7
1.4	Newtons interpolasjonsformel (dividerte differanser)	9
1.5	Interpolasjon ved Vandermonde matrise	16
1.6	Splineinterpolasjon (MAT20x)	18
1.7	Interpolasjon i to dimensjoner (MAT20x)	25
1.8	Praktiske anvendelser	34
1.9	Oppsummering (MAT20x)	34
1.10	Oppsummering - MATLAB (MAT20x)	35
1.11	Kontrollspørsmål	36
1.12	Kontrolloppgaver	36
	Fasit	39
	Forventet læringsutbytte (MAT20x)	

Studentene skal kunne:

- vite hva interpolasjon er,
- vite hva lineær interpolasjon er og kan utføre lineær interpolasjon både analytisk og ved hjelp av MATLAB
- vite hva polynom interpolasjon og kan utføre polynom interpolasjon ved hjelp av MATLAB (*polyval*),
- vite hva spline interpolasjon og kan utføre kubisk spline interpolasjon,
- vite hva interpolasjon i to dimensjoner handler om og kan utføre enkle interpolasjoner ved hjelp av MATLAB gitt x og y datapunkter til å bestemme z_j (z-verdi) i et gitt punkt (x_j, y_j) .

Til eksamen man må kunne:

- utføre lineær interpolasjon gitt 2 punkt og bestemme y_j (y-verdi i gitt x_j),
- gitt spline funksjonen, velge rett polynom til å regne ut y_j (y-verdi i gitt x_j) (se oppgave 1.12.1),
- forklare hva en gitt MATLAB program (se oppgave 1.9.7) utfører (lineær, polynom og kubisk spline).

Vedlegg m-filer (MAT20x):

Eks1Lininterp1.m (eksempel 1.2.1)

Eks1Polyinterp.m (eksempel 1.3.1)

Eks1Splineinterp1.m (eksempel 1.6.1)

Eks1Interp2.m (eksempel 1.7.2)

Eks2interp2a.m, Eks2Lininterp2b.m, Eks2interp2c.m (eksempel 1.7.3)

Anbefalte oppgaver: 1.11.1, 1.12.1, 1.12.2, 1.12.3 og 1.12.7.

Det anbefales å du regner/går gjennom eksempler 1.2.1, 1.3.1, 1.6.1, 1.7.2 (det forventes ikke at du skal skrive MATLAB programmet, men forstår hva programmet utfører, se oppgave 1.12.7).

Kapittel 1

Interpolasjon (MAT20x)

Å approksimere funksjoner er et viktig tema innenfor regneorientert matematikk[†]. Det er logisk at den som skal approksimere funksjonen skal være enklere å jobbe med. Vi lærte tidligere at en (stykkevis) kontinuerlig periodisk funksjon kan tilnærmes til en trigonometrisk rekke (Fourierrekker). Vi lærte også å bestemme taylorrekken som kan være en god tilnærming for funksjonen i et punkt eller i et intervall. Det finnes flere metoder og er viktig å lære når hver metode kan brukes og om deres fordeler og deres begrensninger. I numeriske analyse er interpolasjon en type estimering, en metode for å konstruere nye datapunkter ved å bygge opp funksjoner innenfor et diskret sett med kjente datapunkter. Med andre ord interpolasjon problemet handler om å finne en funksjon, for eksempel ett polynom av grad n , eller et sett av polynomer av grad n definert stykkevis, som reiser gjennom de $n + 1$ målpunktene, $(x_i, y_i), i = 0, \dots, n$.

Tidligere har vi lært å bestemme en tilnærmet funksjonsverdi i et punkt som ligger på en funksjon ved hjelp av tangentlinjen (lineær approksimasjon) eller taylorpolynom. Tangenten eller taylorpolynomet kan brukes som en tilnærmet funksjon for funksjonen nær ett punkt eller på et intervall (konvergenstintervall).

For å sette opp tangentlinjen eller taylorpolynomet måtte vi ha *funksjonstuttrykket* for å bestemme funksjonsderiverte ($f', f'', \dots$). Det vil si man må ha funksjonsderiverte ($f'(x_0), f''(x_0), \dots$) for å bestemme en estimert verdi for $f(x_1)$ der x_1 ligger nær nok x_0 :

$$f(x_1) \simeq f(x_0) + f'(x_0)(x_1 - x_0) + \frac{f''(x_0)}{2!}(x_1 - x_0)^2 + \dots$$

For noen funksjoner er konvergensintervallet ikke så stor, så unøyaktigheten øker når x_1 ikke nær nok x_0 . Når man har målpunkter har man ikke et funksjonsuttrykk og blir umulig å bruke denne approksimeringsmetoden. Taylorrekken er basert på informasjon fra ett punkt på grafen, mens interpolasjon bruker flere punkter til å estimere en funksjonsverdi i et punkt. I noen tilfeller approksimerer Taylorrekken funksjoner nær et punkt og i noen tilfeller kan taylorpolynomet kan være gyldig for alle x -verdier (for eksempel taylorrekken til e^x der konvergensintervallet er alle reelle tall. I mange situasjoner er kanskje ikke funksjonen f kjent, bare dens funksjonsverdier og man kan da bruke interpolasjon.

Hva er interpolasjon? (MAT20x)

Å interpolere (*latin 'stase opp, friske opp, forfalske'*) handler om å bruke matematiske sammenhenger å tilpasse en funksjon gjennom målte verdier. Begrepet er ofte knyttet til at punktet

[†]Oppgavebok-Amir Hashemi

der funksjonen er ukjent, og skal estimeres, ligger innenfor et intervall utspent av punktene der funksjonen er kjent, mens i *ekstrapolasjon*, der punktet ligger utenfor intervallet. Interpolasjon i matematikk handler om å bestemme funksjonsverdien i et punkt ved hjelp av kjente funksjonsverdier i nærliggende punkt. En enkel måte å interpolere er å knytte punktene sammen med rette linjer mellom målepunktene. Men vi ønsker ofte en mer jevnere funksjon i interpolasjonspunktene (knutepunktene, nodene), og dermed man kan bruke polynomer. Det finnes flere interpolasjonsteknikker. Her skal vi fokusere på følgende interpoleringsmetoder:

- Lineær interpolasjon
- Polynom interpolasjon
- Spline interpolasjon

I lineær interpolasjon antar man at det er lineær sammenheng mellom hvert par av datapunktene.

Tanken bak polynominterpolasjon er å bestemme et polynom å passere gjennom målepunktene. En generell parabola har tre frie koeffisienter, og vi bør derfor forvente å kunne tvinge en parabola gjennom tre vilkårlige punkter. Mer generelt, antar at vi har $n + 1$ distinkte tall $\{x_i\}$, $0 \leq i \leq n$ fordelt på et intervall $[a, b]$ der f er definert. Siden et polynom av grad n , $p_n(x)$, har $n + 1$ frie koeffisienter, er det naturlig å prøve å finne et polynom av grad n . Det finnes flere metoder å bestemme slike polynomer. Dividerende (delte) differanser (Newtonemetode) er en rekursiv måte som kan brukes når delintervall lengdene ikke er den samme for alle delintervall sekvensene.

Splineinterpolasjon er en generalisering av lineær interpolasjon. Splineinterpolasjon er en form for interpolering der interpolanten er en spesiell type stykkevis polynom som kalles en spline. Splineinterpolasjon er ofte foretrukket fremfor polynomiskinterpolasjon fordi interpolasjonsfeilen er mindre selv når du bruker lavgradspolynomer for spline. For å beregne stykkevise polynomer, er det viktig å bestemme først hvilket polynombit vi er på. Interpolering ved polynomer er effektiv og robust fordi polynomer er enkle å evaluere, differensiere, og integrere, i forhold til andre valg som kan være trigonometriske og eksponentialfunksjoner.

Det finnes mange interpolasjonsmetoder og det kan da være aktuelt å vurdere følgende for å velge passende interpolasjonsteknikk:

Hvor nøyaktig er denne metoden?

Hvor kostbar og effektiv er metoden med tanke på beregningsoperasjoner?

Hvor glatt er interpolanten?

1.1 Interpolasjon i en dimensjon (MAT20x)

Tenk at vi skal bestemme en funksjonsverdi (y_j) i et punkt (x_j) når noen målte verdier er kjente i et intervall. Der vi kan bruke numeriske metoder for å tilpasse målepunktene med glatte kurver.

Anta det er gitt $n + 1$ (målte) verdier $y_0, y_1, y_n, \dots, y_n$ i distinkte punkter $x_0 < x_1 < \dots < x_n$. Interpolasjonsproblemet er å konstruere en funksjon $p_n(x)$ som går gjennom disse punktene, det vil si vi ønsker å finne en funksjon $p_n(x)$ slik at interpolasjonskravene $p_n(x_i) = y_i$, $0 \leq i \leq n$ tilfredsstilles. Målepunktene $x_0, x_1, \dots$ kalles *interpolasjonspunkt* og $p_n(x)$ kalles *interpolant*, og det å finne en interpolant som går gjennom interpolasjonspunktene kalles *interpolasjon*.

Et enkelt eksempel kan være å bestemme $y(126)$ gitt:

x	80	126	150
y	33	?	68

Her kan vi sette opp en rett linje gjennom de kjente punktene og bestemme $y(125)$:

$$y(126) = y(80) + \frac{68 - 33}{150 - 80}(126 - 80) = 33 + 0.5 \cdot 46 = 56$$

Selv om denne løsningen, nemlig det å anta en lineær sammenheng mellom datapunktene, kan være en løsning av interpolasjonsproblemet, vanligvis (men ikke alltid) er vi interessert i en annen type løsning, for eksempel en jevnere funksjon. Vi spesifiserer derfor alltid en viss klasse av funksjoner fra som vi ønsker å finne en som løser interpolasjonsproblemet. For eksempel, vi kan se etter en funksjon $f(x)$ som er et polynom, et stykkevis glatt polynom, en trigonometrisk funksjon eller en eksponentialfunksjon osv. .

Teorem 1.1.1:

Eksistens og entydighetsteoremet for polynomisk interpolering

(Beviset kreves ikke for MAT20x)

La f være defintert på et lukket begrenset intervall $[a, b]$, og La $x_0, \dots, x_n \in \mathbf{R}$ være distinkte punkter på $[a, b]$, da for et hvert sett av funksjonsverdier, $f(x_0), \dots, f(x_n)$, finnes det bare ett polynom $p_n(x)$ av grad $\leq n$ slik at $p_n(x_i) = f(x_i)$

BEVIS. Vi begynner først med *eksistensdel*. Det kan enkelt bevises ved hjelp av induksjon: For $n = 0$, $p_0(x_i) = f(x_i)$. Anta at p_{n-1} er et polynom av $(n - 1)$ 'te grad og anta at $p_{n-1}(x_i) = f(x_i)$, der $1 \leq i \leq n - 1$. Vi kan bygge opp $p_n(x)$:

$$p_n(x) = p_{n-1}(x) + c(x - x_0)(x - x_1) \cdots (x - x_{n-1})(x - x_n)$$

c må da bestemmes. c kan bli lik null hvis p har lavere grad enn n ($1 \leq i \leq n - 1$). Det blir da å bestemme c for siste interpolasjonskravet: $p_n(x_i) = f(x_i)$.

$$c = \frac{f(x_n) - p_{n-1}(x_n)}{\prod_{i=0}^{n-1} (x - x_i)}$$

Da er det bevist *eksistensdelen* av teoremet.

For *entydighetsdelen*, la to polynomer $p(x)$ og $q(x)$ interpolerer disse punktene. Da

$$r_n(x) = p_n(x) - q_n(x)$$

har n null punkter, unntatt at begge er lik null. Begge tilfredsstiller interpoleringskravene:

$$r_n(x_i) = (p_n - q_n)(x_i) = 0, \quad 0 \leq i \leq n$$

som tilsier r har $n + 1$ nullpunkter som kommer motsier vår antagelse, og dermed vi kan konkludere:

$$p(x) \equiv q(x)$$

Entydighetsdelen er også dermed bevist.

MATLAB - kort oversikt (lineær, polynom, spline interpolasjon (MAT20x))

For å bruke MATLAB til å interpolere, skal man sette opp målepunktene/datapunktene som matriser og bruker deretter kommandoer som utfører interpolasjonen. Dette kan hjelpe oss til å bestemme et ny funksjonsverdi gitt et punkt ligger i intervallet datapunktene ligger i.

For *lineær interpolasjon* kan man bruke kommandoen der y -verdien til x_j (y_j) blir estimert ved linear interpolasjon:

$y_j = \text{interp1}(x, y, x_j, 'linear')$, der datapunktene x og y angis som rekke vektorer. For å tegne grafen kan man diskretiser interpoleringsintervallet:

```
n=length(x);
xleft = min(x);
xright = max(x);
np = 100*n;
% deler opp interpolasjonsintervallet for x i np deler
dx = (xright - xleft)/(np-1);
xi = xleft:dx:xright;
yi =interp1(x,y,xi,'linear');
plot(x,y,'o',xi,yi,'black')
```

For *polynominterpolasjon* kan bruke $p = \text{polyfit}(x, y, n)$ der koeffisientene til en n 'te grads funksjon bestemmes og kan bruke $\text{polyval}(p, x_j)$ til å bestemme polynometsverdi i x_j ($p(x_j)$) som er estimert verdi til $y_j = y(x_j) = p(x_j)$.

```
% Eks1polyinterp.m utfører polynom interpolasjon ved hjelp av polyfit og polyval
clc;
clf;
x=[-1,0,1,2];
y=[3,-4,5,-6];
pn = polyfit(x,y,3)
xj=1.5;
yj=polyval(pn,xj)

plot(xj,yj,':*')

hold on;

n=length(x);
xleft = min(x);
xright = max(x);
np = 100*n;
% deler opp interpolasjonsintervallet for x i np deler
dx = (xright - xleft)/(np-1);
xi = xleft:dx:xright;
yi =polyval(pn,xi);

plot(x,y,'o',xi,yi,'black')
title('polynom interpolasjon')
```

For *spline interpolasjon* kan man bruke kommandoen der y -verdien til x_j blir estimert ved spline interpolasjon: $y_i = \text{interp1}(x, y, x_i, 'spline')$, der datapunktene x og y angis som rekke vektorer. For å tegne grafen kan man diskretiser interpoleringsintervallet vist i eksempelet.

```
%Eks1Splineinterp1.m fil utfører kubisk spline interpolasjon
clear;
clc;
clf;
x=[-1,0,1,2];
y=[3,-4,5,-6];
m=length(x); % Tegner grafen for m x-verdier
xleft = min(x);
xright = max(x);
mp = 100*m; % Tegner grafen for m x-verdier
dx = (xright - xleft)/(mp - 1);
xi = xleft:dx:xright;

yi =interp1(x,y,xi,'spline');
plot(x,y,'o',xi,yi,'black')
title('Lineær interpolation')

hold on;

xj=1.5;
yj=interp1(x,y,xj,'spline')
plot(xj,yj,'*')
```

For interpolasjon i *to dimensjoner* kan man bruke

$z_i = \text{interp2}(\mathbf{x}, \mathbf{y}, \mathbf{A}, x_i, y_i, \text{metode})$ der z_i blir estimert ved hjelp av x_i og y_i der metode i forhold til vårt behov i faget kan velges:

$\mathbf{x}$ og $\mathbf{y}$ er rekkematriser og har dimensjon $(1 \times n)$ og $(1 \times m)$ henholdsvis, og $\mathbf{A}_{m \times n}$ inneholder funksjonsverdiene for til datapunktene.

'linear': Lineær interpolasjon fra nærmeste punktene
'spline': Kubisk splineinterpolasjon

1.2 Lineær interpolasjon (MAT20x)

I matematikk er lineær interpolering en metode for kurvepassing ved bruk av lineære polynomer (linjer) for å konstruere nye datapunkter innenfor et diskret sett med kjente datapunkter.

Lineær interpolasjon handler om å finne lineære interpolanter i intervaller mellom hvert par av målte verdier $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. Linjen mellom $(x_0, y_0), (x_1, y_1)$ kan skrives som

$$p(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$$

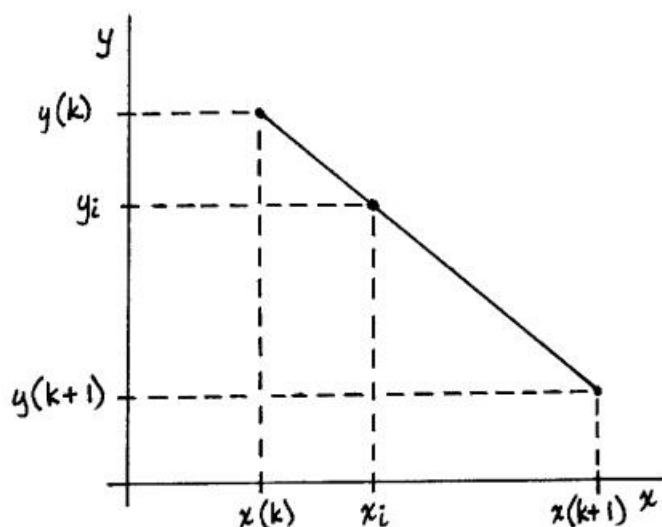
For å estimere y_j som ligger mellom disse, kan vi da regne ut:

$$y_j \simeq p(x_j) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x_j - x_0)$$

Lineær interpolasjon tar utgangspunkt at det er lineær sammenheng mellom de to kjente målepunktene. The Newton interpolasjonsformel for lineær interpolasjon er

$$p(x) = y_k + \frac{y_{k+1} - y_k}{x_{k+1} - x_k}(x - x_k) .$$

y_i kan da bestemmes ved: $y_i = y(x_i) = y_k + \frac{y_{k+1} - y_k}{x_{k+1} - x_k}(x_i - x_k)$



Vi skal se senere nærmere på Lagranges og Newtons formel for polynom interpolasjon der vi antar det er en n 'te grad går gjennom de $(n + 1)$ datapunktene.

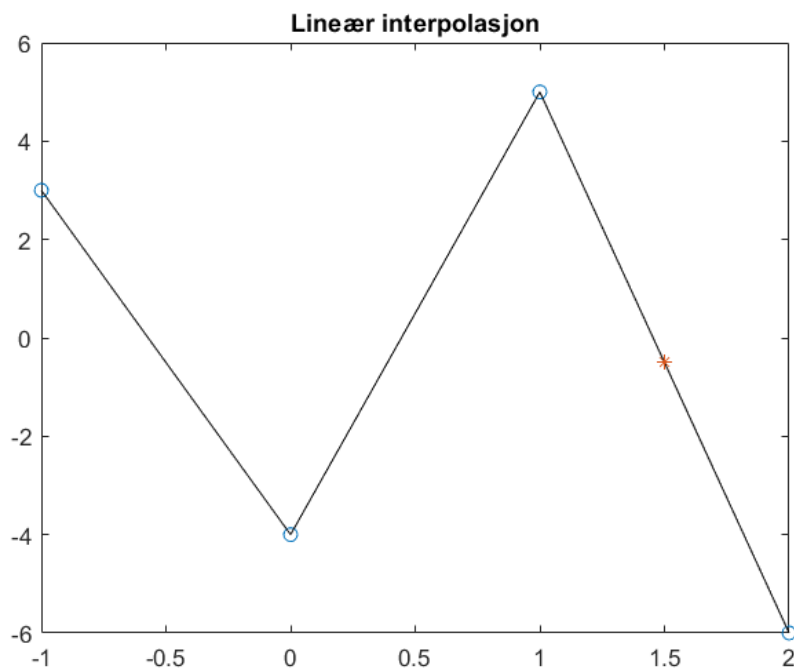
Eksempel 1.2.1: Gitt målepunkter:

x	-1	0	1	2
y	3	-4	5	-6

Bestem $y(1.5)$ ved lineær interpolasjon ved hjelp av Matlab.

Løsning: x - og y -verdiene leses inn som rekke matriser i matlab program som bruker `yi=interp(x,y,xi, 'metode')` kommandoen til å bestemme $y_i = y(x_i)$.

```
%Eks1Lininterp1.m fil
clear;
clc;
clf;
x=[-1,0,1,2];
y=[3,-4,5,-6];
n=length(x);
xleft = min(x);
xright = max(x);
np = 100*n;
% deler opp interpolasjonsintervallet for x i np deler
dx = (xright - xleft)/(np-1);
xi = xleft:dx:xright;
yi =interp1(x,y,xi,'linear');
plot(x,y,'o',xi,yi,'black')
title('Lineær interpolation')
xj=1.5;
hold on;
yj=interp1(x,y,xj,'linear')
plot(xj,yj,'*')
>>-0.5
```



1.3 Polynomisk interpolasjon (MAT20x)

Med $n + 1$ målepunkter vil det være mulig å tilpasse ett polynom av grad minst lik n gjennom disse. Det ble vist tidligere teoremet eksistens og entydighet til interpolasjonsproblemet med polynomer. Det er flere måter å bygge opp slike polynomer.

Eksempel 1.3.1: Gitt datapunkter:

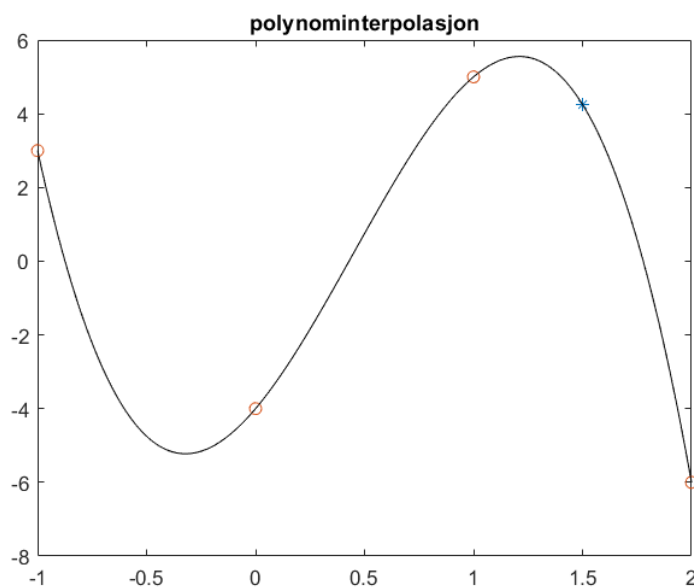
x	-1	0	1	2
y	3	-4	5	-6

Bestem $y(1.5)$ ved polynominterpolasjon ved hjelp av Matlab. Tegn grafen til polynomet og det interpolerte punktet i samme koordinatssystem.

Løsning: x - og y -verdiene leses inn som rekke matriser i matlab program som bruker *polyval* kommandoen til å bestemme $y(x_j)$.

```
% Eks1polyinterp.m utfører polynominterpolasjon ved hjelp av polyfit og polyval
clc; clf;
x=[-1,0,1,2];
y=[3,-4,5,-6];
pn = polyfit(x,y,3)
xj=1.5;
yj=polyval(pn,xj)
plot(xj,yj,'*')
hold on;
n=length(x);
xleft = min(x);
xright = max(x);
np = 100*n;
% deler opp interpolasjonsintervallet for x i np deler
dx = (xright - xleft)/(np-1);
xi = xleft:dx:xright;
yi =polyval(pn,xi);

plot(x,y,'o',xi,yi,'black')
title('polynom interpolasjon')
>> pn =    -6.0000    8.0000    7.0000   -4.0000
>> yj =     4.2500
```



1.4 Newtons interpolasjonsformel (dividerte differanser)

I polynominterpolasjon tilnærmingen bestemmes ved at vi krever at feilen skal være null i noen isolerte punkter. Et velkjent eksempel er sekanten til en funksjon f . Da velger vi to forskjellige punkter x_0 og x_1 og lar tilnærmingen p_1 være den rette linja (det polynomet av grad 1) som har samme verdi som f i disse punktene. Med andre ord krever vi at

$$p_{i+1}(x_i) = p_j(x_j) = y_i, \quad i = 0, 1, 2, \dots, n-1$$

Nå kan vi anta at vi har $n+1$ punkter, $x_0, x_1, \dots, x_n$, and $n+1$ funksjonsverdier, $y_0, y_1, \dots, y_n$. Vi ønsker å finne ett polynom, $p_n(x)$, av grad n , har spesifiserte verdier ($\{y_i\}_{i=0}^n$) i spesifiserte punkt ($\{x_i\}_{i=0}^n$):

$$p_n(x_k) = y_k \quad \text{for } k = 0, \dots, n. \quad (1.1)$$

Det forutsettes at datapunktene er distinkte, $x_i \neq x_k$ if $i \neq k$. Det finnes da ett polynom (*eksistens og entydighetsteoremet*) som kan bygges opp ved hjelp av disse data punktene og forskjellige metoder fører til samme polynom.

For å bygge opp interpolasjonspolynomet kan vi bruke Newtons interpolasjonsformelen. Denne rekursive metoden kan ta inn flere og flere punkt for å bygge opp det interpolerende polynomet. dett første Interpolant polynomet Metoden bygger opp en polynom serie og hver gang interpolerer et nytt ekstra punkt. $p_0(x)$, har grad null, det vi si en konstant. p_0 tilfredsstillter første interpolasjonsbetingelsen: (1.1) for $k = 0$:

$$p_0 = y_0 \quad .$$

Deretter finner vi neste polynomet, $p_1(x)$, av grad 1 som interpolerer både y_0 og y_1 . We choose $p_1(x)$ of the form

$$p_1(x) = p_0(x) + (x - x_0) \cdot c_1$$

Denne tilfredsstillter interpolasjonskravet i x_0 ($p_1(x_0) = p_0(x_0)$) $p_1(x_0) = p_0(x_0) = y_0$. Det neste interpolasjonskravet $p_1(x_1) = y_1$, hjelper til å bestemme c_1 :

$$y_1 = y_0 + (x_1 - x_0) \cdot c_1 \quad ,$$

$$c_1 = \frac{y_1 - y_0}{x_1 - x_0} \quad , \quad (1.2)$$

$$p_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0} (x - x_0) \quad . \quad (1.3)$$

Formelen (1.2) beregner koeffisienten c_1 (*dividerte differanser eller delte differanser*). Nå kan vi sette opp $p_2(x)$ ved hjelp av $p_1(x)$:

$$p_2(x) = p_1(x) + (x - x_1)(x - x_0)c_2 \quad .$$

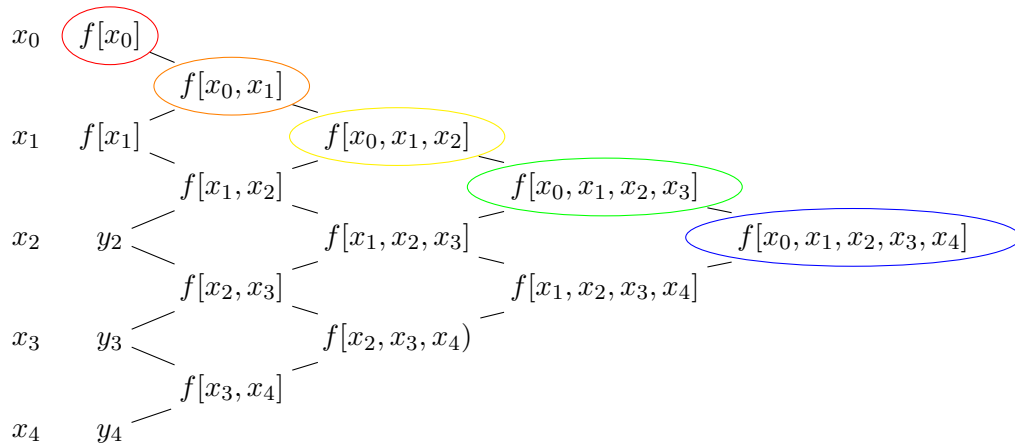
Denne oppbygningen tilfredsstillter interpolasjonskravene for x_1 ($p_2(x_1) = p_1(x_1)$), samt at interpolasjonskravet i x_0 er bevart. Nå kan vi bestemme c_2 ved hjelp av kravet: $p_2(x_2) = y_2$:

$$c_2 = \frac{\frac{y_2 - y_0}{x_2 - x_0} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_1} \quad .$$

Ved å fortsette denne metoden, kan vi bytte opp det allmenne leddet for polynomet. Hvis polynom av grad $j-1$ tilfredsstillter interpolasjonskravene (1.1) for $k < j$, da kan vi prøve å finne $p_k(x)$ ved induksjon:

$$p_i(x) = p_{i-1}(x) + (x - x_{i-1}) \cdots (x - x_0)c_i \quad , \quad (1.4)$$

og kan da bruke interpolasjonskravet: $p_j(x_j) = f_j$ og kan da bestemme c_j . Hvis $p_{j-1}(x)$ har grad $j-1$, da gir (1.4) p_j grad j siden andre ledd har j faktorene som inneholder x . Hvis vi fortsetter på samme måte, kan da bygge opp c_n som kan hjelpe oss å sette opp $p_n(x)$ som tilfredsstiller interpolasjonskravene. (1.1). Denne metoden kalles for Newtons interpolasjonsformel.



$$\begin{aligned}
 f[x_i] &= f(x_i) = f_i \\
 f[x_i, x_{i+1}] &= \frac{f[x_{i+1}, x_i] - f[x_i]}{x_{i+1} - x_i} \\
 &\vdots \\
 f[x_i, x_{i+1}, \dots, x_{i+k}] &= \frac{f[x_{i+1}, x_{i+2}, \dots, x_{i+k}] - f[x_i, x_{i+1}, \dots, x_{i+k-1}]}{x_{i+k} - x_i}
 \end{aligned}$$

Newtons interpolasjon:

$$\begin{aligned}
 f(x) &= f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2) \\
 &\quad + f[x_0, x_1, x_2, x_3, x_4](x - x_0)(x - x_1)(x - x_2)(x - x_3) + \dots \\
 &\quad + f[x_0, x_1, x_2, \dots, x_n](x - x_0)(x - x_1)(x - x_2) \dots (x - x_{n-1}) + \dots
 \end{aligned}$$

Delte differanser (dividerte differanser) er symmetriske med hensyn til argumentene, dvs. uavhengig av rekkefølgen på argumentene.

Eksempel 1.4.1: Gitt datapunktene:

i	0	1	2	3
x	-1	0	1	2
y	3	-4	5	-6

a) Bruk datapunktene nedenfor til å bestemme et polynom av tredjegrad ($p_3(x)$) ved hjelp av Newtons interpolasjonsformel:

Bestem en estimertverdi for ut $y(1.5)$ ved hjelp av $p_3(x)$.

b) Sett opp et MATLAB program som bruker Newtonsformel til å regne ut $y(1.5)$.

(det kreves ikke at man skal kunne det matematiske grunnlaget for Newtons formel, og det forventes ikke at man skal lage avanserte MATLAB program, men det er fint at man kan se/gjøre seg kjent med hvordan dette kan gjøres med utgangspunkt i matematikken)

Løsning:

a)

$$\begin{array}{lcl}
 x_0 = -1 & y_0 = f[x_0] = 3 & \\
 & & \swarrow \\
 & & f[x_0, x_1] = \frac{-4 - 3}{0 - (-1)} = -7 \\
 & & \swarrow \quad \searrow \\
 x_1 = 0 & y_1 = f[x_1] = -4 & f[x_0, x_1, x_2] = 8 \\
 & & \swarrow \quad \searrow \\
 & & f[x_1, x_2] = 9 \quad f[x_0, x_1, x_2, x_3] = -6 \\
 & & \swarrow \quad \searrow \\
 x_2 = 1 & y_2 = f[x_2] = 5 & f[x_1, x_2, x_3] = -10 \\
 & & \swarrow \quad \searrow \\
 & & f[x_2, x_3] = 11 \\
 x_3 = 2 & y_3 = f[x_3] = -6 &
 \end{array}$$

$$\begin{aligned}
 f[x_0, x_1] &= \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{-4 - 3}{0 - (-1)} = \frac{-4 - 3}{0 - (-1)} = -7 \\
 f[x_0, x_1, x_2] &= \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{9 - (-7)}{1 - (-1)} = 8 \\
 f[x_0, x_1, x_2, x_3] &= \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} = \frac{-10 - (-8)}{2 - (-1)} = -6
 \end{aligned}$$

$$p_3(x) = \sum_{j=0}^3 f[x_0, x_1, \dots, x_j] \prod_{i=0}^{j-1} (x - x_i)$$

$$p_3(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$$

$$p_3(x) = 3 - 7(x + 1) + 8(x + 1)x - 6(x + 1)x(x - 1)$$

$$p_3(x) = -6x^3 + 8x^2 + 7x - 4$$

♦

b) MATLAB program

```
function pn = newtoninterpol(x,y,xi)
n=length(x);
a(1)=y(1);
for k=1:n-1
    d(k,1)=(y(k+1)-y(k))/(x(k+1)-x(k));
end
for j=2:n-1
    for k=1:n-j
        d(k,j)=(d(k+1,j-1)-d(k,j-1))/(x(k+j)-x(k));
    end
end
for j=2:n
    a(j)=d(1,j-1);
end
xn=vpa(x);
d=vpa(d)
a=vpa(a);
clear x
syms x
%
Df(1)=vpa(1);
c(1)=a(1);
for j=2:n
    Df(j)=(x-xn(j-1)).*Df(j-1);
    c(j)=a(j).*Df(j);
end
format short
% Setter opp polynomet
p=sum(c)
% Forenkler polynomet
p=simplify(sum(c))
% Bestemmer pn(xi)
f(1)=vpa(1);
c(1)=a(1);
for j=2:n
    Df(j)=(xi-xn(j-1)).*Df(j-1);
    c(j)=a(j).*Df(j);
end
format short
pn=sum(c);
```

```
% Eksempel
clc;
x=[-1,0,1,2];
y=[3,-4,5,-6];
newtoninterp(x, y,1.5)
>>
d =
[ -7.0,   8.0,  -6.0]
[  9.0, -10.0,   0]
[ -11.0,   0,   0]
p =8.0*x*(1.0*x + 1.0) - 7.0*x - 6.0*x*(x - 1.0)*(1.0*x + 1.0) - 4.0
p =- 6.0*x^3 + 8.0*x^2 + 7.0*x - 4.0
ans = 4.25
```

Lagranges interpolasjonsformel

En annen måte å bygge opp interpolasjonsproblemet er Lagranges interpolasjonsformel:

$$m_k(x) = \prod_{i \neq k, i=0}^n (x - x_i) , \quad (1.5)$$

$$L_k(x) = m_k(x)/m_k(x_k) . \quad (1.6)$$

Nevneren i (1.6) blir ikke null da x_j er aldri lik x_k if $j \neq k$. Polynomer $m_k(x)$ og $l_k(x)$ har grad n siden produktene (1.5) har n ledd, og videre $m_k(x_i) = l_k(x_i) = 0$ if $i \neq k$. This is the purpose of the formula (1.5). Normalisering (1.6) medfører $l_k(x_k) = 1$. Interpolasjonsproblemet $p_n(x)$ kan uttrykkes ved hjelp av $l_k(x)$ som:

$$p_n(x) = \sum_{k=0}^n y_k L_k(x) = \sum_{k=0}^n y_k \prod_{i \neq k, i=0}^n \frac{(x - x_i)}{(x_k - x_i)} , \quad (1.7)$$

Hvis x er en av interpolasjonspunktene, for eksempel $x = x_m$, er da (1.7) med $k = m$ fra forskjellige fra null. Så (1.7) tilfredsstiller interpolasjonskravene (1.1). 1.7 som tar utgangspunkt i (5) og (6) kalles *Lagrange interpolasjonsformel*.

Eksempel 1.4.2: a) Bruk datapunktene nedenfor til å bestemme et polynom av tredjegrad ($p_3(x)$) ved hjelp av Lagrange interpolasjonsformel:

i	0	1	2	3
x	-1	0	1	2
y	3	-4	5	-6

♦

Regn ut $p_3(1.5)$.

b) Et MATLAB program for langrange interpolasjonen er gitt ved:

```
function Lagrange(x,y,n,xx)
%
%   Lagranges interpolasjonsformel
%   LAGRANGE(X,Y,N,XX)
%   x og y er rekke vektorer og n antall punkt (i=j=n) der
%   X (i=1,n) og Y(j=1,n) og Lagranges metoden er brukt til å bestemme
%   y-verdien til xx.
%Vi kan skrive et skript på MATLAB som vi kan kalle for eksempel "$Eks_Lagrange.m$":
clc;
    sum=0;
    for i = 1:n
        produkt=y(i);
        for j=1:n
            if (i~=j)
                produkt= produkt*(xx-x(j))/(x(i)-x(j));
            end
        end
        sum = sum +produkt
    end
    yy =sum
    plot(x,y,'o-')
    title('Lagrange Interpolasjon ')
    xlabel('x - value')
    ylabel('y - value')
    axis([min(x)-1 max(x)+1 0 max(y)+2])
    grid on
    hold on
    plot(xx,yy,'*', 'MarkerSize', 10)
end
```

Regn ut $p(1.5)$ ved hjelp MATLAB programmet.

c) Bruk programmet og datapunktene gitt nedenfor til å bestemme en estimertverdi for $y(5)$.

i	0	1	2	3	4
x	1	2	4	7	8
y	4	5	10	9	16

Løsning:

a) Vi bruker $p_3(-1) = 3$, $p_3(0) = -4$, $p_3(1) = 5$ og $p_3(2) = -6$ til å bygge opp Lagrange polynomial:

$$L_{n,k}(x) = \prod_{i=0, i \neq k}^3 \frac{(x - x_i)}{(x_k - x_i)}$$

Da har vi:

$$L_{3,0}(x) = \frac{(x - x_1)(x - x_2)(x - x_3)}{(x_0 - x_1)(x_0 - x_2)(x_0 - x_3)} = \frac{(x - 0)(x - 1)(x - 2)}{(-1 - 0)(-1 - 1)(-1 - 2)} = \frac{x(x^2 - 3x + 2)}{(-1)(-2)(-3)} = -\frac{1}{6}(x^3 - 3x^2 + 2x)$$

$$L_{3,1}(x) = \frac{(x - x_0)(x - x_2)(x - x_3)}{(x_1 - x_0)(x_1 - x_2)(x_1 - x_3)} = \frac{(x + 1)(x - 1)(x - 2)}{(0 + 1)(0 - 1)(0 - 2)} = \frac{(x_2 - 1)(x - 2)}{(1)(-1)(-2)} = \frac{1}{2}(x^3 - 2x^2 - x + 2)$$

$$L_{3,2}(x) = \frac{(x - x_0)(x - x_1)(x - x_3)}{(x_2 - x_0)(x_2 - x_1)(x_2 - x_3)} = \frac{(x + 1)(x - 0)(x - 2)}{(1 + 1)(1 - 0)(1 - 2)} = \frac{x(x_2 - x - 2)}{(2)(1)(-1)} = -\frac{1}{2}((x^3 - x^2 - 2x))$$

$$L_{3,3}(x) = \frac{(x - x_0)(x - x_1)(x - x_2)}{(x_3 - x_0)(x_3 - x_1)(x_3 - x_2)} = \frac{(x + 1)(x - 0)(x - 1)}{(2 + 1)(2 - 0)(2 - 1)} = \frac{x(x^2 - 1)}{(3)(2)(1)} = \frac{1}{6}(x^3 - x)$$

$$p_3(x) = \sum_{k=0}^3 y_k \cdot L_{3,k}(x) = y_0 L_{3,0}(x) + y_1 L_{3,1}(x) + y_2 L_{3,2}(x) + y_3 L_{3,3}(x)$$

$$p_3(x) = (3) - \frac{1}{6}(x^3 - 3x^2 + 2x) + (-4)\frac{1}{2}(x^3 - 2x^2 - x + 2) + 5(-\frac{1}{2}((x^3 - x^2 - 2x))) + 6(\frac{1}{6}(x^3 - x))$$

$$p_3(x) = (-\frac{1}{2} - 2 - 5 + 2 - 1)x^3 + (\frac{3}{2} + 4 + \frac{5}{2})x^2 + (-1 + 2 + 5 + 1)x - 4 = -6x^3 + 8x^2 + 7x - 4$$

$$p(1.5) = 4.25$$

b)

```
clc;
clc;
x=[-1,0,1,2]
y=[3,-4,5,-6]
Lagrange(x,y,4,1.5)
ans = 4.25
```

c) **MATLAB-skript:**

```
clc;
clc;
x=[1,2,4,7,8];
y=[4,5,10,9,16];
Lagrange(x,y,5,2.3)
```

Nå er det bare å kjøre eksempelet:

```
>>Eks_Lagrange.m
sum =
    5.8701
yy =
    5.8701
```

1.5 Interpolasjon ved Vandermonde matrise

Den tredje metoden er den mest generelle. Denne metoden kan være effektiv når tidligere metoder. I denne metoden kan interpoleringsproblemet utgjøres som et system av lineære ligninger der man kan bruke algebraiske metoder. Vi har allerede vist at polynomial interpoleringsmetode har en entydig løsning. The fact that the interpolating polynomial is unique falls out immediately as a consequence of what we already know.

Vandermonde Matrise

Interpolasjons polynomet $p_n(x)$ skrives som:

$$p_n(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n .$$

der $a_0, a_1, a_2, \dots, a_n$ er reelle koeffisienter. Ved å anta at polynomet går gjennom datapunktene (1.1) får vi: form

$$\begin{aligned} a_0 + a_1x_0 + \cdots + a_nx_0^n &= y_0 \\ a_0 + a_1x_1 + \cdots + a_nx_1^n &= y_1 \\ &\vdots \\ a_0 + a_1x_n + \cdots + a_nx_n^n &= y_n . \end{aligned}$$

Interpolasjonsproblemet kan da skrives på matrise form:

$$\mathbf{V}\mathbf{a} = \mathbf{y} , \tag{1.8}$$

der

$$\mathbf{V} = \begin{pmatrix} 1 & x_0 & \cdot & \cdot & \cdot & x_0^n \\ 1 & x_1 & \cdot & \cdot & \cdot & x_1^n \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 1 & x_n & \cdot & \cdot & \cdot & x_n^n \end{pmatrix}, \quad \mathbf{a} = \begin{pmatrix} a_0 \\ a_1 \\ \cdot \\ \cdot \\ \cdot \\ a_n \end{pmatrix}, \quad \mathbf{f} = \begin{pmatrix} y_0 \\ y_1 \\ \cdot \\ \cdot \\ \cdot \\ y_n \end{pmatrix} .$$

Matrisen $\mathbf{V}$ kalles *Vander Monde* matrise ($(n+1) \times (n+1)$) som ikke er singulær og dermed inverterbar. Dermed eksisterer interpolasjonspolynomet og er en-entydig. (1.8).

$\mathbf{V}$ er en ikke-singulær kvadratisk matrise og dermed inverterbar. Dermed interpolasjonspolynomet eksisterer og er en-entydig (1.8). Men denne metoden er mer regnekrevende enn de andre to metodene (Newton og Lagrange) selv om MATLAB utfører regningen.

Eksempel 1.5.1: (MAT20x)

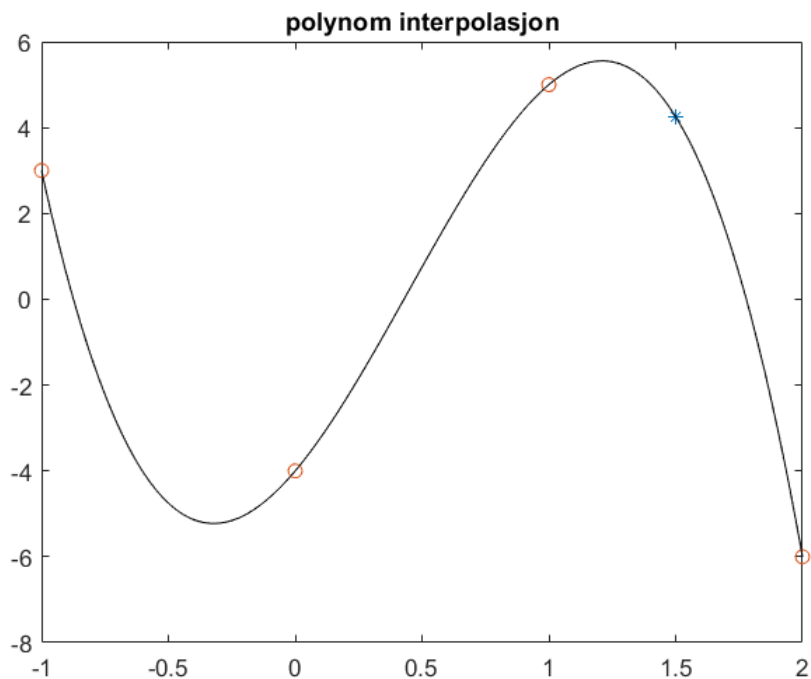
Bruk datapunktene nedenfor til å bestemme et polynom av tredjegrad ($p_3(x)$) ved hjelp av MATLAB. Bruk interpolasjonspolynomet til å bestemme funksjonsverdien for $x_j = 1.5$.

i	0	1	2	3
x	-1	0	1	2
y	3	-4	5	-6

$$\textbf{Løsning: } p_3(x) = -6x^3 + 8x^2 + 7x - 4, \quad p(1.5) = 4.25$$

MATLAB - program:

```
% Eks1Polyinterp.m
clc; clf;
x=[-1; 0; 1; 2];
y =[3 ; -4 ; 5; 6];
M=[x.^0 x.^1 x.^2 x.^3];
a=M\y
% a er koeffisientene til det interpolerende polynomet
% legg merke til at polynomet her bygges opp slik: a(1) + a(2)*t + a(3)*t.^2+ a(4)*t.^3;
plot(x, y,'or')
hold on;
t=linspace(-1,2);
p=a(1) + a(2)*t;
hold on;
t=linspace(-1,2);
p=a(1) + a(2)*t + a(3)*t.^2+ a(4)*t.^3;
plot(t,p,'black')
%Ellers man kan bruke polyfit
q=polyfit(x,y,3)
% legg merke til at polynomet her bygges opp slik: r=q(4) + q(3)*t + q(2)*t.^2+ q(1)*t.^3;
polyval(q,1.5)
>>polyfitvandinterp
q =
    -6.0000     8.0000     7.0000    -4.0000
ans =
     4.2500
```



1.6 Splineinterpolasjon (MAT20x)

Splineinterpolering er en form for interpolasjon der interpolanten er et stykkvis polynom, kalt en spline. Splineinterpolasjon er en generalisering av lineær interpolasjon der interpolerende polynomer skal være glatte i knutepunktene. Vi skal altså ikke finne ett polynom for hele interpolasjonsintervallet, men et sett av polynomer, og målet med prosedyren er å finne parametrene til disse polynomene. Spline metodene går på å finne ett polynom per delintervall og sette dem sammen. Spline interpolering er ofte foretrukket over polynomial interpolering fordi interpolasjonsfeilen er mindre selv om man bruker lavgradspolynom for spline.

En funksjon $S(x)$ er spline av grad k på $[a, b]$ der $x_0 = a \leq x_1 \leq x_2 \leq \dots \leq x_n = b$ og $S \in C^{k-1}$ (klasse $k - 1$, $k - 1$ ganger kontinuerlig deriverbar).

$$S(x) = \begin{cases} S_1(x) & \text{når } x_0 = a \leq x \leq x_1 \\ S_2(x) & \text{når } x_1 \leq x \leq x_2 \\ \vdots & \vdots \\ S_n(x) & \text{når } x_{n-1} \leq x \leq x_n = b \end{cases}$$

Kubisk splineinterpolasjon (MAT20x)

En kubisk spline S er et stykkevis kubisk polynom definert på et intervall $[a, b]$ slik at S, S' og S'' er kontinuerlige på hele intervallet. I kubisk spline er overgangene mellom polynomialstykkene er glatte, og det kan vises at (vi skal ikke vise her):

Gitt datapunktene La S være kubisk spline som interpolerer en funksjon f på knutepunktene $\{x_i\}_{i=1}^n$. Kubisk spline er glatteste interpolasjon blant alle interpolanter $p \in C^2[a, b]$. Beviset går på å vise $\int_a^b S''(x)dx \leq \int_a^b f''(x)dx$.

Interpolasjonskravene for kontinuitet og kontinuitet av første og andre derivert er da:

$$S(x) \in C^2[a, b] : \begin{cases} S_i(x) = S_{i-1}(x) \\ S'_i(x) = S'_{i-1}(x) \\ S''_i(x) = S''_{i-1}(x) \end{cases}, i = 1, 2, \dots, n-1$$

(Det kreves ikke mer enn dette i MAT20x.)

Kontinuitetskravet tilbyr $2n$ ligninger, første derivert og andre derivert kravene gir $2(n-1)$ ligninger og dermed har man totalt $2n + 2(n-1) = 4n - 2$ ligninger med $4n$ ukjente. Dermed har vi 2 frihetsgrad. En *naturlig kubisk spline* er definert ved $S''(x_0) = S''(x_n) = 0$.

La oss nå interpolasjonspolynomet være naturlig kubiske spline gitt som:

$$p_i(x) = A_i(x - x_i)^3 + B_i(x - x_i)^2 + C_i(x - x_i) + D_i \quad \text{for } x_i \leq x \leq x_{i+1}.$$

Vi ønsker å bestemme koeffisientene A_i, B_i, C_i og D_i punktene $\{(x_i, y_i)\}_{i=0}^n$. La $d_i = x_{i+1} - x_i$ og $z_i = S''(x_i)$. Ved å bruke de tre kravene og naturlig kubisk spline betingelsene kan man sette opp et system av $n-1$ lineære likninger uttrykt ved:

$$d_{j-1}z_{j-1} + 2(d_{j-1} + d_j)z_j + d_jz_{j+1} = \frac{3}{d_j}(y_{j+1} - y_j) - \frac{3}{d_{j-1}}(y_j - y_{j-1})$$

$$\begin{pmatrix} 1 & 0 & 0 & & & \\ d_1 & 2(d_1 + d_2) & d_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & 0 & d_{n-2} & 2(d_{n-3} + d_{n-2}) & d_{n-1} & \\ & & 0 & 0 & 1 & \end{pmatrix} \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_{n-2} \\ z_{n-1} \end{pmatrix} = 6 \begin{pmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_{n-2} \\ \lambda_{n-1} \end{pmatrix},$$

der $d_i = \Delta x_i = x_{i+1} - x_i$, $\Delta y_i = y_{i+1} - y_i$, $h_i = \frac{\Delta y_i}{d_i}$, $\lambda_i = h_{i+1} - h_i$, $i = 0, 1, 2, \dots, n_1$ og ved hjelp av z_i kan vi finne koeffisientene i hvert polynom ved formlene:

$$\begin{aligned} A_i &= \frac{z_{i+1} - z_i}{6h_i} \\ B_i &= \frac{z_i}{2} \\ C_i &= \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2z_i + z_{i+1})}{6} \\ D_i &= y_i. \end{aligned}$$

MATLAB - kubisk spline (MAT20x)

Eksempel 1.6.1: (MAT20x)

Bruk datapunktene nedenfor til å bestemme et polynom av ($y(1.5)$) ved hjelp av splineinterpolasjon:

i	0	1	2	3
x	-1	0	1	2
y	3	-4	5	-6

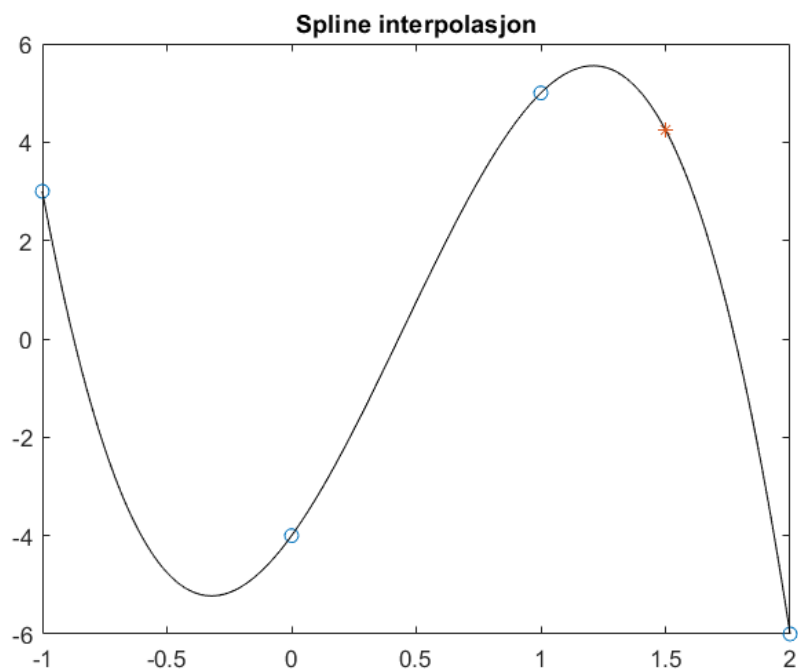
♦

MATLAB programmet for interpolasjon:

```

% Eks1Lininterp1.m fil
clear;
clc;
clf;
x=[-1,0,1,2];
y=[3,-4,5,-6];
n=length(x);
xleft = min(x);
xright = max(x);
np = 100*n;
dx = (xright - xleft)/(np - 1);
xi = xleft:dx:xright;
yi =interp1(x,y,xi,'spline');
plot(x,y,'o',xi,yi,'black')
title('Spline interpolasjon')
xj=1.5;
hold on;
yj=interp1(x,y,xj,'spline')
plot(xj,yj,'*')

```



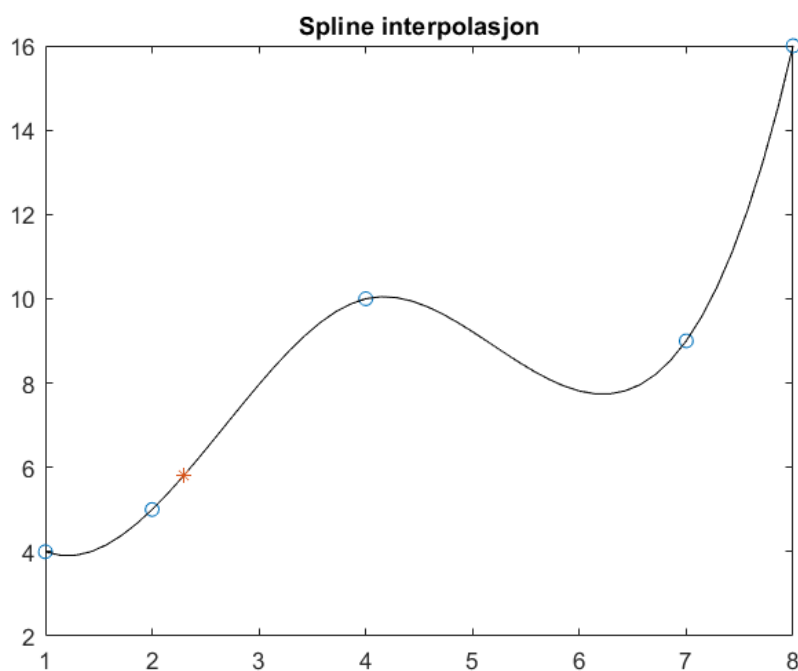
Eksempel 1.6.2: (MAT20x)

Bruk datapunktene nedenfor til å bestemme $y(2.3)$ ved hjelp av MATLAB spline interpolasjon:

i	0	1	2	3	4
x	1	2	4	7	8
y	4	5	10	9	16

Løsning:

```
% Eks2Lininterp1.m fil
clear;
clc;
clf;
x=[1,2,4,7,8]
y=[4,5,10,9,16]
n=length(x);
xleft = min(x);
xright = max(x);
np = 10*n;
dx = (xright - xleft)/(np - 1);
xi = xleft:dx:xright;
yi =interp1(x,y,xi,'spline');
plot(x,y,'o',xi,yi,'black')
title('Spline interpolasjon')
xj=2.3;
hold on;
yj=interp1(x,y,xj,'spline')
plot(xj,yj,':*')
>>
yj = 5.8157
```



◆

Eksempel 1.6.3: (MAT20x)

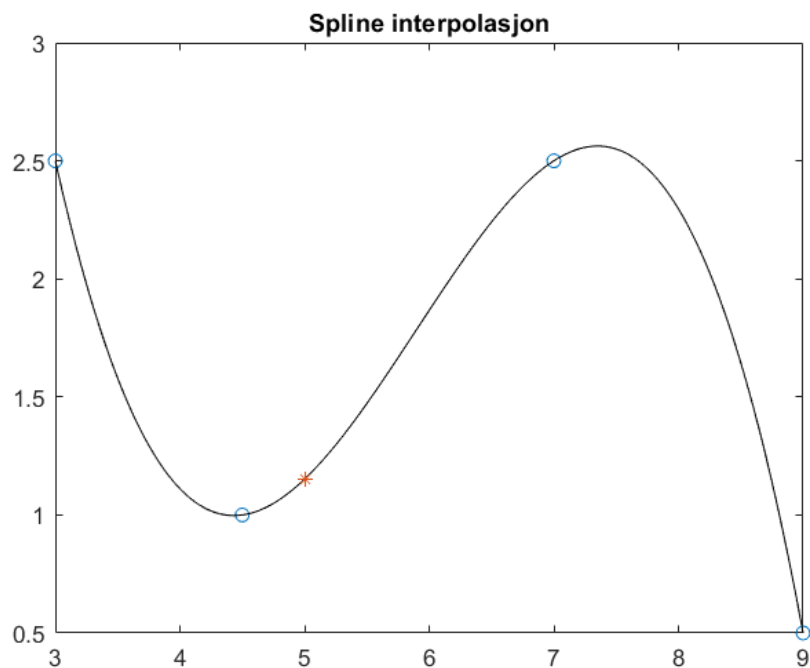
Bruk datapunktene nedenfor til å bestemme $y(5)$ ved spline metoden ved hjelp av MATLAB:

i	0	1	2	3
x_i	3.0	4.5	7.0	9.0
y_i	2.5	1.0	2.5	0.5

◆

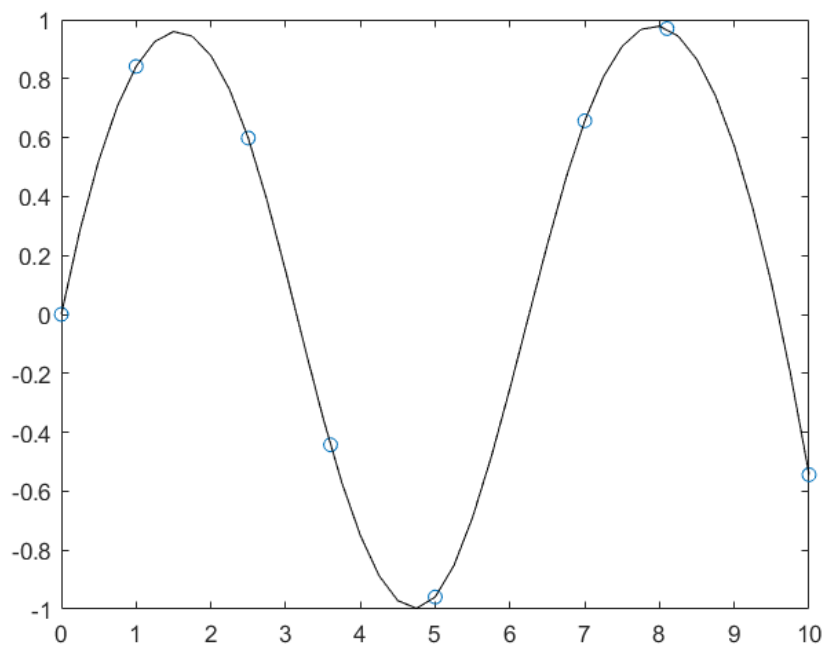
Løsning:

```
% Eks3Splineinterp1.m fil
clc; clf;
x=[3.0 4.5 7.0 9.0];
y=[2.5 1.0 2.5 0.5];
n=length(x);
xleft = min(x);
xright = max(x);
np = 100*n;
dx = (xright - xleft)/(np - 1);
xi = xleft:dx:xright;
yi =interp1(x,y,xi,'spline');
plot(x,y,'o',xi,yi,'black')
title('Spline interpolasjon')
xj=5;
hold on;
yj=interp1(x,y,xj,'spline')
plot(xj,yj,'*')
>>
yi =
    1.1519
```



Eksempel 1.6.4: Bruk spline interpolasjon for $y = \sin x$ der $x = 0, 1, 2.5, 3.6, 5, 7, 8.1, 10$ til å bestemme tilsvarende y -verdier og tegn grafen til den interpolerte funksjonen sammen med punktene.

```
% Eks4Splineinterp1.m fil
clear;
clc;
clf;
x = [0 1 2.5 3.6 5 7 8.1 10];
y = sin(x);
xx = 0:.25:10;
yy = spline(x,y,xx);
plot(x,y,'o',xx,yy,'black')
```



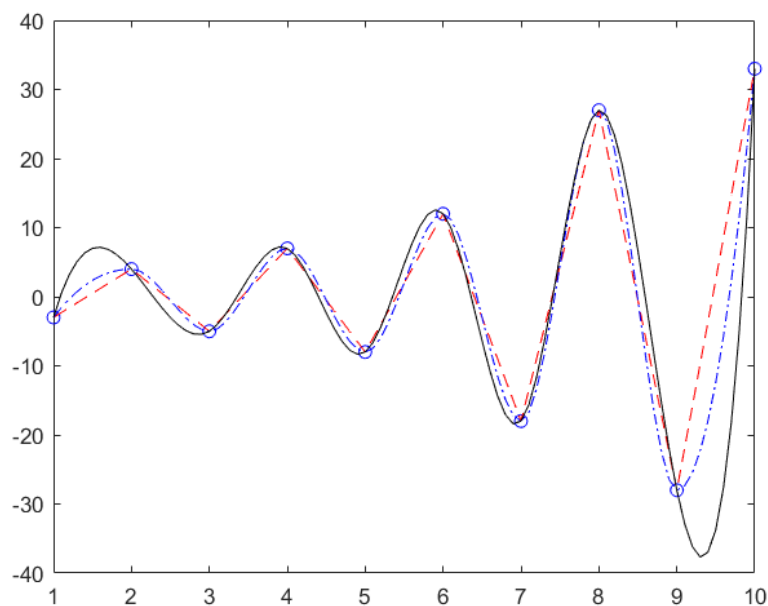
Oppsummeringseksempel med MATLAB (MAT20x)

I neste eksempel tar vi lineær, kubisk spline og spline interpolasjon med et MATLAB eksempel.

Eksempel 1.6.5: Bruk datapunktene

$x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$, $y = [-3, 4, -5, 7, -8, 12, -18, 27, -28, 33]$
til å interpolere x_i som ligger på intervallet $0 \leq x_i \leq 10$ med steg lengde 0.1 med MATLAB (lineær og kubisk spline).

```
function [y1, y2] = Interpolator();  
clc; clf;  
  
x=[ 1 2 3 4 5 6 7 8 9 10];  
y=[-3 4 -5 7 -8 12 -18 27 -28 33];  
  
plot(x,y,'ob')  
xi=1:0.1:10;  
  
method=char('linear','spline');  
col={'--r','-k'};  
  
for k=1:2  
    yi{k}=interp1(x,y,xi,method(k,:));  
  
    hold on;  
    plot(xi,yi{k},col{k});  
end  
  
y1=yi{1};  
y2=yi{2};
```



◆

1.7 Interpolasjon i to dimensjoner (MAT20x)

Interpolasjonsmetodene for en enkelt variabel sak kan generaliseres til flere dimensjoner. Vi begrenser oss her med interpolering i to variabler, når en i en antall rutenettpunkter i xy-planet har gitt høydeverdier $z(x_i; y_j)$. Med passende interpolering ønsker vi å beregne høydeverdier z for mellomliggende x - og y -verdier.

Følgende interpolasjonsmetoder er:

- bilinear interpolation
- bikubisk interpolasjon
- bikubisk spline

Den enkleste metoden er bilineær interpolasjon.

Bilineær interpolasjon metoden for $z_i = f(x_i, y_i)$:

$$f(x_i, y_i) = \frac{a + b + c + d}{(x_2 - x_1)(y_2 - y_1)}$$

der

$$a = f(x_1, y_1)(x_2 - x_i)(y_2 - y_i)$$

$$b = f(x_2, y_1)(x_i - x_1)(y_2 - y_i)$$

$$c = f(x_1, y_2)(x_2 - x_i)(y_i - y_1)$$

$$d = f(x_2, y_2)(x_i - x_1)(y_i - y_1)$$

Eksempel 1.7.1: (MAT20x: Fint om studentene selv leser gjennom)

La oss ta et enkelt eksempel med interpolasjon gjennom:

$$(3, 2, 5)(5, 2, 10)(3, 6, 3) \text{ og } (5, 6, 9)$$

der vi skal bestemme $z(x = 3.5, y = 4.9)$

Løsning:

Vi har fått koordinatene:

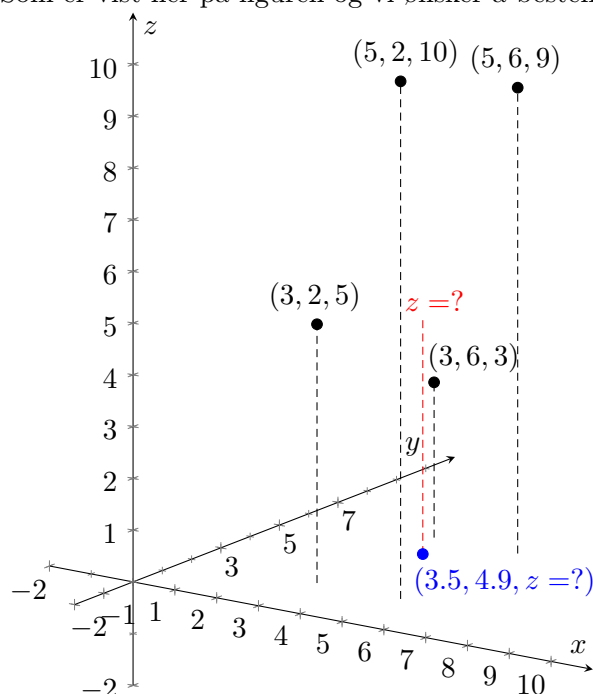
$$z(x_1 = 3, y_1 = 2) = 5,$$

$$z(x_2 = 5, y_1 = 2) = 10,$$

$$z(x_1 = 3, y_2 = 6) = 3,$$

$$z(x_2 = 5, y_2 = 6) = 9.$$

Som er vist her på figuren og vi ønsker å bestemme z - verdien i punktet $(x = 3.5, y = 4.9, z = ?)$.



Bilineær interpolasjon innebærer at lineær interpolasjon gjennomføres i både x - og y -retning. Det vil si om utfører interpolasjon en om gangen. Lineinterpolasjon i x -retning ved $y = y_1 = 2$ medfører:

$$z(x = 3.5, y_1) = z(x_1, y_1) + \frac{z(x_2, y_1) - z(x_1, y_1)}{x_2 - x_1}(x - x_1) = 5 + \frac{10 - 5}{5 - 3}(3.5 - 3) = 5 + 1.25 = 6.25$$

og lineær interpolation i x -retning ved $y = y_2 = 6$ gir:

$$z(x = 3.5, y_2) = z(x_1, y_2) + \frac{z(x_2, y_2) - z(x_1, y_2)}{x_2 - x_1}(x - x_1) = 3 + \frac{9 - 3}{5 - 3}(3.5 - 3) = 3 + 1.5 = 4.5$$

Nå tar vi lineær interpolasjon i y -retning:

$$z(3.5, 4.9) = z(3.5, y_1) + \frac{z(3.5, y_2) - z(3.5, y_1)}{y_2 - y_1}(y - y_1) = 6.25 + \frac{4.5 - 6.25}{6 - 2}(4.9 - 2) = 4.98125 \blacklozenge$$

MATLAB for interpolasjon i to dimensjoner

$\mathbf{z}_i = \text{interp2}(\mathbf{x}, \mathbf{y}, \mathbf{A}, \mathbf{x}_i, \mathbf{y}_i, \text{metode})$ bestemmer matrisen $\mathbf{z}_i$ inneholder elementer som tilsvarer elementene i $\mathbf{x}_i$ og $\mathbf{y}_i$ og bestemmes ved interpolering i den todimensjonale funksjonen som er spesifisert av matriser $\mathbf{x}_{1 \times n}$, $\mathbf{y}_{1 \times n}$ og $\mathbf{A}_{m \times n}$ kan definere datapunktene. Alle interpolasjonsmetoder krever at $\mathbf{x}$ og $\mathbf{y}$ er monoton. Bruk metodene 'linear', 'cubic', 'spline' eller 'nearest' for å få raskere interpolering når $\mathbf{x}$ og $\mathbf{y}$ er like og monotone.

$\mathbf{x}_i$ og $\mathbf{y}_i$ kan være matriser, i hvilket tilfelle `interp2` bestemmer verdiene til $\mathbf{z}_i$ som tilsvarer punktene $(\mathbf{x}(i, j), \mathbf{y}(i, j))$. Alternativt kan man passere henholdsvis rad- og kolonnevektorene $\mathbf{x}_i$ og $\mathbf{y}_j$. Metode kan vi velge:

- 'linear' for bilinær interpolering
- 'spline' for kubisk splineinterpolasjon

Eksempel 1.7.2: (MAT20x)

Gitt: $x = [0, 1, 2]$; $y = [10, 11, 12]$; $A = [10, -1, 11; 7, 5, 2; 1, 4, 5]$ Som beskriver datapunktene:

$(0, 10, 10), (0, 10, -1), (0, 10, 11), (1, 11, 7), (1, 11, 5), (1, 11, 2), (2, 12, 1), (2, 12, 4), (2, 12, 5)$.

Bruk lineær og spline interpolasjonsmetoden på (x_i, y_i) til å bestemme z_i der $0 \leq x \leq 2$ og $10 \leq y \leq 12$ ved hjelp av MATLAB.

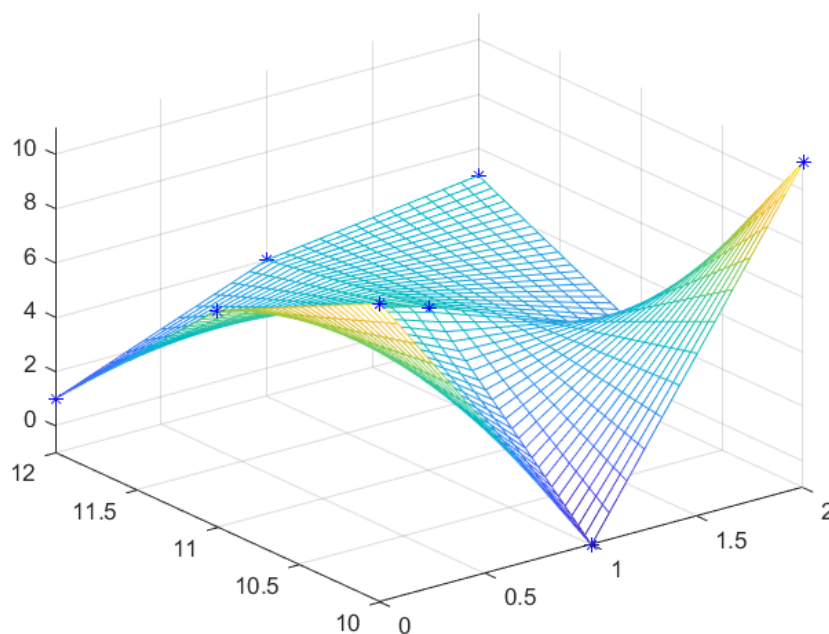
Hint: Bruk `interp2(x,y,A,xi,yi,'metode')`, der metode kan velges *Linear* og *Spline*. Legg merke til at x_i og y_i skal diskretiseres i n og m antall deler ved hjelp av:

$xi = \text{linspace}(\min(x), \max(x), n)$ og $yi = \text{linspace}(\min(y), \max(y), m)$.

Løsning:

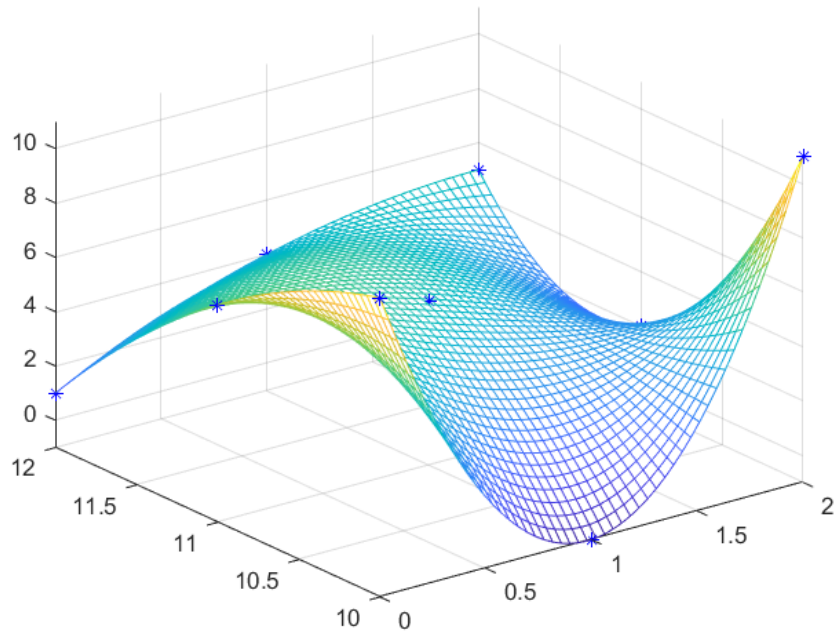
a) Lineær interpolasjon

```
% Eks1Lininterp2.m fil
clf; colormap("default");
x = [0,1,2];
y = [10,11,12];
A = [10,-1,11;7,5,2;1,4,5];
xi = linspace(min(x), max(x), 25);
yi = linspace(min(y), max(y), 50)';
zi=interp2(x,y,A,xi,yi, "Linear");
mesh(xi,yi,zi);
[x,y] = meshgrid(x,y);
hold on;
plot3(x,y,A,"b*");
hold off;
```



b) Spline interpolasjon

```
% Eks1Splineinterp2.m fil
clf;
colormap ("default");
x = [0,1,2];
y = [10,11,12];
A = [10,-1,11;7,5,2;1,4,5];
xi = linspace (min (x), max (x), 50);
yi = linspace (min (y), max (y), 50)';
mesh (xi,yi,interp2 (x,y,A,xi,yi, "Spline"));
[x,y] = meshgrid (x,y);
hold on;
plot3 (x,y,A,"b*");
hold off;
```



◆

Eksempel 1.7.3: (MAT20x)

Gitt: $x = [-4, 4]$; $y = [-4, 4]$; $z = \sqrt{x^2 + y^2}$ beskriver datapunktene (x, y, z)

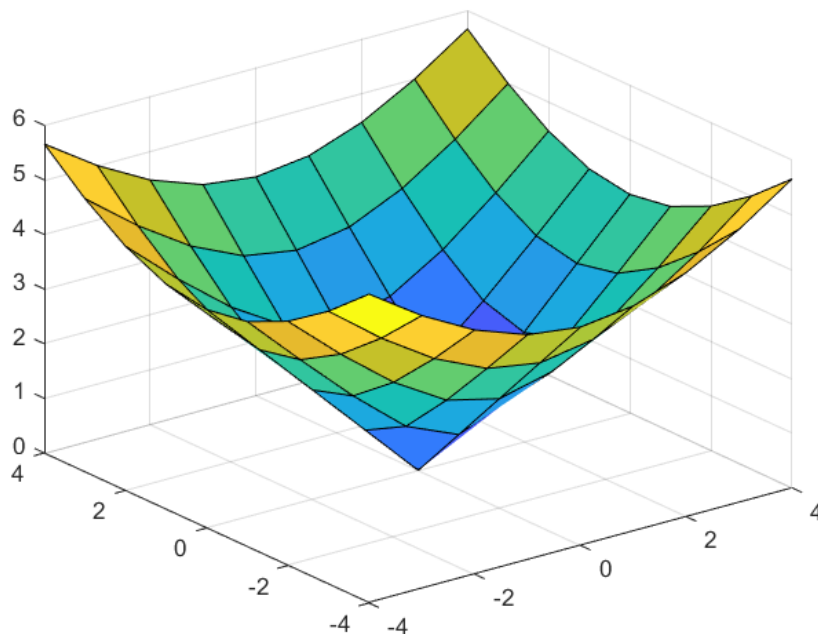
Bruk lineær og spline interpolasjonmetoden på (x_i, y_i) til å bestemme z_i der $-4 \leq x \leq 4$ og $10 \leq y_i \leq 12$ ved hjelp av MATLAB.

Hint: Bruk `interp2(x,y,A,xi,yi,'metode')`, der metode kan velges *linear/bilinear* og *spline/bicubic*. Legg merke til at x_i og y_i skal diskretiseres i n og m antall deler ved hjelp av: $x_i = \text{linspace}(\min(x), \max(x), n)$ og $y_i = \text{linspace}(\min(y), \max(y), m)$.

Løsning:

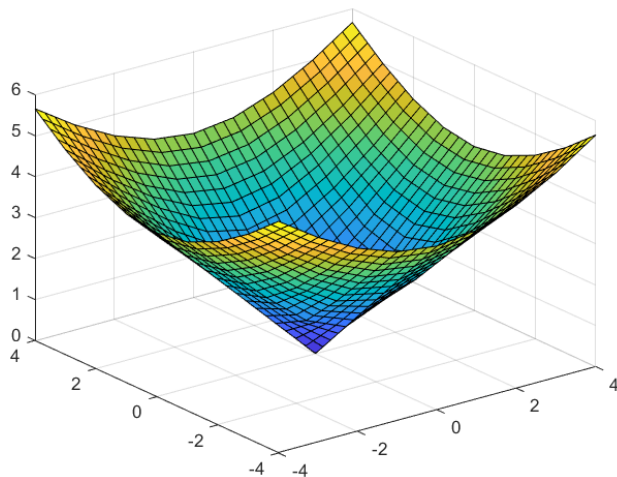
Grafen til funksjonen:

```
% Eks2interp2a.m fil
clf;
[x,y] = meshgrid(-4:1:4);
z = sqrt(x.^2+y.^2);
surf(x,y,z)
```



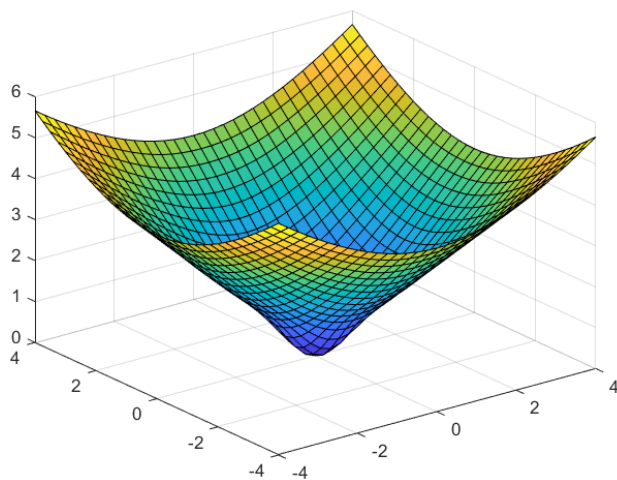
Bilineær/lineær interpolasjon

```
% Eks2Bilininterp2b.m fil
clf; [x,y] = meshgrid(-4:1:4);
z = sqrt(x.^2+y.^2);
[xi,yi] = meshgrid(-4:0.25:4);
zi1 = interp2(x,y,z,xi,yi,'bilinear');
surf(xi,yi,zi1);
```



bicubic/Spline interpolasjon

```
% Eks2cubicinterp2c.m fil
clf; [x,y] = meshgrid(-4:1:4);
z = sqrt(x.^2+y.^2);
[xi,yi] = meshgrid(-4:0.25:4);
surf(xi,yi,zi2);
zi2 = interp2(x,y,z,xi,yi,'bicubic');
surf(xi,yi,zi2);
```

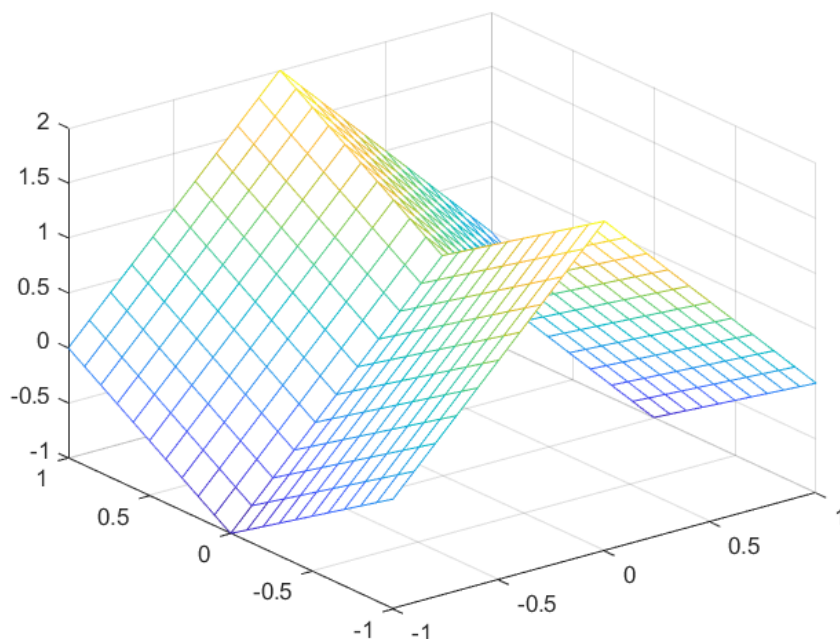


◆

Interpolasjon i tre dimensjoner

$\mathbf{v}_i = \text{interp3}(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{A}, \mathbf{x}_i, \mathbf{y}_i)$; Området kan defineres ved $\mathbf{x}_{1 \times n}$, $\mathbf{y}_{1 \times m}$ $\mathbf{z}_{1 \times p}$ $\mathbf{A}$ har da dimensjon $m \times n \times p$. Metodene 'linear', 'cubic', 'spline' er de mest brukte interpolasjonsmetodene.

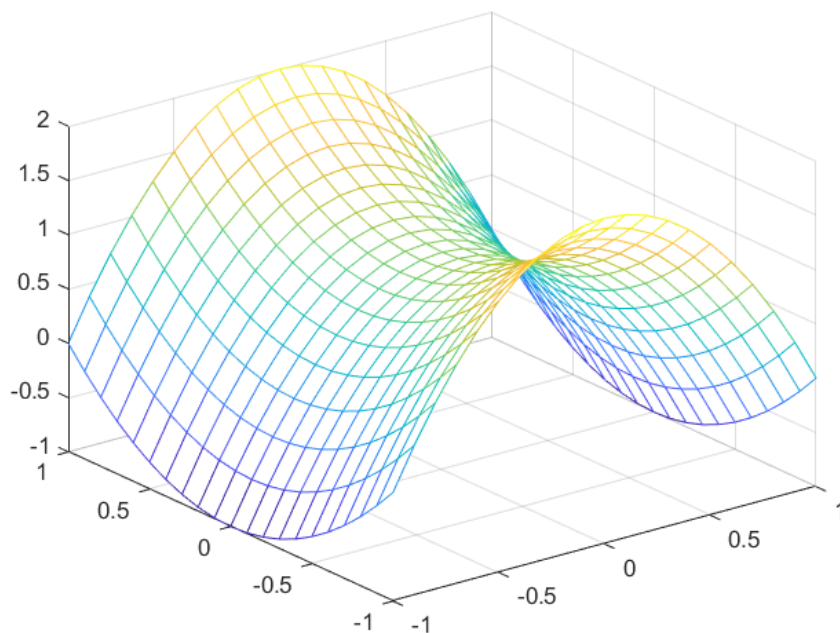
```
% EksLinInterp3.m
clf;
x = -1:1;
y = -1:1;
z = -1:1;
f = @(x,y,z) x.^2 - y - 2*z.^2;
[xx, yy, zz] = meshgrid (x, y, z);
v = f (xx,yy,zz);
xi = -1:0.1:1;
yi = -1:0.1:1;
zi = -1:0.1:1;
[xxi, yyi, zzi] = meshgrid (xi, yi, zi);
vi = interp3(x, y, z, v, xxi, yyi, zzi, "linear");
[xxi, yyi, zzi] = ndgrid (xi, yi, zi);
vi2 = interpn (x, y, z, v, xxi, yyi, zzi, "linear");
mesh (zi, yi, squeeze (vi2(1,:,:)));
```



```

% EksSplineInterp3.m
clf;
x = -1:1;
y = -1:1;
z = -1:1;
f = @(x,y,z) x.^2 - y - 2*z.^2;
[xx, yy, zz] = meshgrid (x, y, z);
v = f (xx,yy,zz);
xi = -1:0.1:1;
yi = -1:0.1:1;
zi = -1:0.1:1;
[xxi, yyi, zzi] = meshgrid (xi, yi, zi);
vi = interp3(x, y, z, v, xxi, yyi, zzi, "spline");
[xxi, yyi, zzi] = ndgrid (xi, yi, zi);
vi2 = interpn (x, y, z, v, xxi, yyi, zzi, "spline");
mesh (zi, yi, squeeze (vi2(1,:,:)));

```

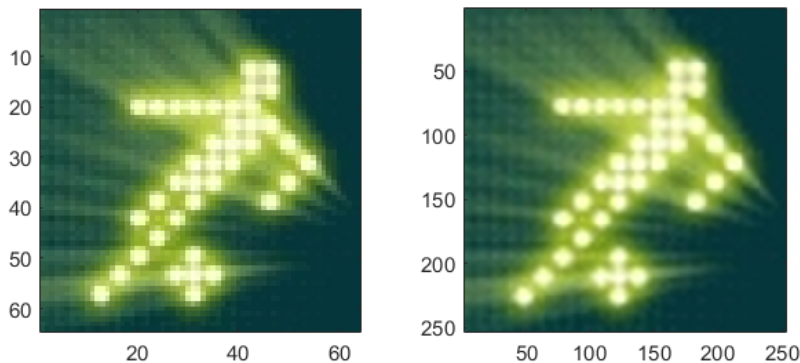


1.8 Praktiske anvendelser

Interpolering er nyttig når du må skalere ting opp eller ned.

```
Eksempel 1.8.1: function Bildebehandling()
% Eksempel på praktisk anvendelse av interp2
% Bicubic smoothing

src = imread('bilde_src.jpg');
size(src)
[x,y] = meshgrid(1:64);
[xi,yi] = meshgrid(1:0.25:64);
for k = 1:3
    bilde_spline(:,:,k) = ...
        interp2(x,y,double(src(:,:,k)),xi,yi,'spline');
end
subplot(1,2,1);
image(src);
axis square;
subplot(1,2,2);
image(uint8(bilde_spline));
axis square;
```



♦

1.9 Oppsummering (MAT20x)

I *interpolering* passerer interpolanten gjennom datapunktene mens i *approksimasjon/tilnærning*, som begrepet antyder, approksimanten passerer nært datapunktene. Når det gjelder hensiktsmessighet, er interpolering hensiktsmessig å bruke for å utjevne datapunkter med støy (støy i målinger) og ikke passende når datapunkter er utsatt for eksperimentelle feil. På den annen

side er approksimasjon i hovedsak passende for beregning av spesielle funksjoner. Hvorfor? For utformingen av disse funksjonene er det bare viktig at den estimerte funksjonen er i nærheten av den nøyaktige underliggende matematiske funksjonen og ikke viktig at funksjonsverdiene samsvarer nøyaktig på et bestemt punkt. Så approksimanten er gyldig nær et punkt eller på et intervall. For å bestemme approksimanten da man ofte trenger informasjon om funksjonen og dens deriverte, andre deriverte, osv. Det er ofte i eksperimenter man ikke har en funksjon å forholde seg til, men man har tilgang til målte verdier i noen måle punkter og da man kan sette opp datapunkter. I interpolasjon skal den målrettede funksjonen passere gjennom alle gitte datapunkter og man ønsker å finne funksjonsverdien til et mellompunkt i datapunktene, mens vi i lineær kurvepassing finner den generelle trenden til avhengig variabel. Når man passer den approksimerte kurven til datapunktene, definerer man en kostandsfunksjon som skal minimeres for å få den minste feilen.

I *interpolering* tvinger man interpolasjonsfunksjonen å passere gjennom punktene, mens i *kurvetilpasning* går ut på å bruke matematiske metoder, for eksempel minste kvadratiske metoden, å fastlegge en teoretisk kurve som best mulig følger en gitt eksperimentell kurve basert på empiriske data. Det er derfor i kurvetilpasning har man begrepet residual (feilledet/restleddet) som skal minimeres for å passe på at den teoretiske kurven er en god tilnærming (nærmest) til datapunktene.

Polynomisk interpolasjon er den enkleste og vanligste typen interpolasjon. Det er vist at det finnes ett unikt polynom i n -te grad som passerer gjennom $(n+1)$ datapunkter. Det er mange måter å beregne eller representere ett polynom, men de koker ned til det samme polynomet. Lagranges og Newtons interpolasjonsformel, presenterer her, er to av de interpolasjonsteknikkene. Det er numerisk kostbar å få et polynom passere gjennom stort antall datapunkter som sannsynligvis vil gi utilfredsstillende svingende oppførsel i interpolanten. Stykkevis polynominterpolasjon (splineinterpolasjon) kan løse noe av komplikasjonene. lineær interpolasjon antar at det er lineær sammenheng mellom to nærmeste nabo punkt. Feilen vil da vokse dess større krumning funksjonen har mellom interpolasjonspunktene. Ulempen kan også være at interpolanten er ikke deriverbar i nodene. I splineinterpolasjon finner med en familie av polynomer som er jevne i nodene (datapunktene). En av vanligste splineinterpolasjoner er *naturlig kubisk interpolasjon*.

1.10 Oppsummering - MATLAB (MAT20x)

Interpolasjon i en dimensjon med MATLAB definerer man datapunktene som x - og y -matriser (rekke vektorer):

$$x = [x_0, x_1, x_2, \dots, x_n];$$

$$y = [y_0, y_1, y_2, \dots, y_n];$$

I lineær interpolasjon og kubisk spline interpolasjon bruker man:

$$y_i = \text{interp1}(x, y, x_i, 'metode');$$

der y_i bestemmes med denne kommandoen for et gitt x_i som ligger i x -intervallet, og metode kan velges *lineær* eller *spline* (det finnes flere metoder også, men vi holder oss til disse to metodene, se eksempel 1.2.1 og 1.6.1).

For interpolasjon i to dimensjoner kan man bruke kommandoen

$$z_i = \text{interp2}(\mathbf{x}, \mathbf{y}, \mathbf{A}, x_i, y_i)$$

Legg merke til at x_i og y_i kan defineres som vektorer hvis man ønsker å interpolere et intervall (se eksempel 1.7.2 der datapunktene tilpasses til kommandoen).

1.11 Kontrollspørsmål

Oppgave 1.11.1:

- Hva handler interpolasjon om?
- Hva er lineær interpolasjon? Kan du nevne flere interpolasjonsteknikker?
- Kan flere polynomer interpolere ett visst antall datapunkt? (du kan henvise til teorem)
- Hva er spline interpolering? Hva er fordelene til interpolering?

1.12 Kontrolloppgaver

Oppgave 1.12.1:

Vertikal hastigheten til en raket $v(t)$ (m/s) ved tiden t (s) er gitt i tabellen nedenfor:

t (s)	0	10	15	20	22.5	30
$v(t)$ (m/s)	0	227.04	362.78	517.35	602.97	901.6

- Interpoler $v(16)$ ved hjelp av punktene $t = 15$ og $t = 20$.
- Sett opp et lineært uttrykk for $v(t)$ for intervallet $15 \leq t \leq 20$.

Oppgave 1.12.2:

- Gitt datapunktene $(a, f(a))$ og $(b, f(b))$. Sett opp et uttrykk for polynominterpolasjonen ved Lagranges interpolasjonsformel.
- Gitt datapunktene:

x	15	18	22
y	24	37	25

Bruk Lagrange polynominterpolasjon for å finne en estimertverdi for $x = 16$

- Sett opp et lineært uttrykk for $v(t)$ for intervallet $15 \leq t \leq 20$.

Oppgave 1.12.3:

Følgende logaritmisk tabell er gitt:

x	4.0	4.5	5.5	6.0
$f(x)$	0.6020600	0.6532125	0.7403627	0.7781513

- Interpoler $\log(5)$ ved hjelp av punktene $x = 4$ og $x = 6$.
- Interpoler $\log(5)$ ved hjelp av punktene $x = 4.5$ og $x = 5.5$.
- Interpoler $\log(5)$ ved hjelp av punktene $x = 4$, $x = 5.5$ og $x = 6$.
- Gitt den eksakte verdien $\log(5) = 0.69897$. Bestem relativ avvik for disse interpolasjoner. Bruk dette til å konkludere noe.

Oppgave 1.12.4:

Gitt datapunktene (x_i, y_i) :

x	0	2	3	4
y	7	11	28	63

- Bruk Lagrangsmetoden til å sette opp
- Bruk dette til å regne ut $y(2.5)$
- Skriv en MATLAB code som utfører del b)

Oppgave 1.12.5:

Gitt datapunktene (x_i, y_i) :

x	-2	-1	0	1	2
$f(x)$	2	14	4	2	2

- a) Bruk Newtonsметoden til å sette opp interpolasjonspolynomet av fjerde grad ($p_4(x)$).
- b) Bruk dette til å regne ut $y(1.5)$
- c) Skriv et MATLAB program som utfører del a) og b)

Oppgave 1.12.6:

Gitt datapunktene (x_i, y_i) :

i	0	1	2	3
x_i	0	2	3	5
y_i	2	4	5	82

- a) Bruk Newtonsметoden (dividerte differanser) til å sette opp interpolasjonspolynomet ($p_3(x)$) ved hjelp av de gitt datapunkter.
- b) Bruk dette til å regne ut $y(2.5)$
- c) Skriv et MATLAB program som utfører del a) og b)

Oppgave 1.12.7:

Et MATLAB skript er git som følgende:

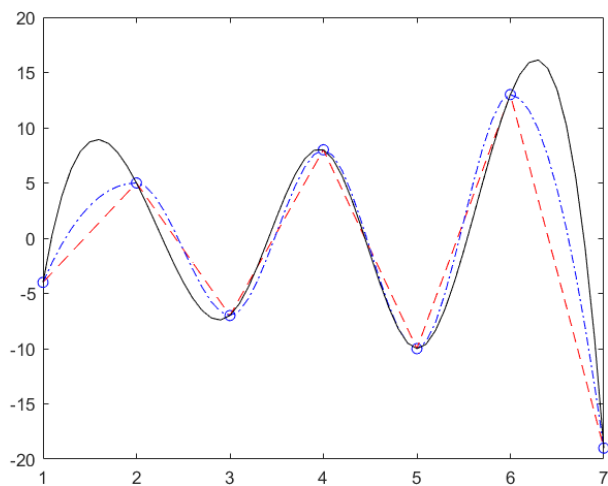
```
function [y1, y2, y3] = Interpolator()
clc;
clf;
x=[ 1 2 3 4 5 6 7 ]
y=[-4 5 -7 8 -10 13 -19]

plot(x,y,'ob')
xi=1:0.1:7

method=char('linear','xxxx','yyyyy')
col={'--r','-b','-k'}

for k=1:3
yi{k}=interp1(x,y,xi,method(k,:))
hold on;
plot(xi,yi{k},col{k})
end

y1=yi{1}
y2=yi{2}
y3=yi{3}
```



- Forklar kort hva programmet utfører.
Hva skal stå for xxxx og yyyyy?. Begrunn svaret.
- Kan du forklare kort hvilken av disse tre interpolasjonspolynomene er glattest og si kort hvorfor.

Fasit

Oppgave 1.11.1:

- a) Interpolasjon handler om å bestemme en estimert funksjonsverdi for en x -verdi ved å bruke kjente funksjonsverdier i nærliggende punkt.
- b) Lineær interpolasjon eller to punkt interpolasjon antar det er lineær sammenheng mellom to nærliggende datapunkter. Polynom interpolasjon og spline interpolering er blant de kjente integrasjonsteknikker.
- c) Nei, kun ett polynom (teorem for eksistens og en-entydighet til interpolasjonspolynomer).
- d) spline-interpolasjon en form for interpolering der interpolanten er en spesiell type stykkevis polynom som kalles en spline. Spline-interpolering er ofte foretrukket fremfor polynomisk interpolering fordi interpolasjonsfeilen er liten selv når man bruker polynomier med lav grad til spline.
- a) Lineær interpolasjon: $v(t) = v(t_0) + (t - t_0) \cdot v[t_1, t_0] = v(15) + \frac{v(20) - v(15)}{(20 - 15)} = 362.78 + 30.914 \simeq 393.694$
- b) Lineær interpolasjon: $v(t) = v(15) + \frac{f(20) - f(15)}{(20 - 15)}(t - 15) = 362.78 + 30.914(t - 15) \simeq 100.93 + 30.914t$

Oppgave 1.12.2:

- a)
- b) Lagrange:

$$L_{2,0}(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - 18)(x - 22)}{(15 - 18)(15 - 22)}$$

$$L_{2,1}(x) = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x - 15)(x - 22)}{(18 - 15)(18 - 22)}$$

$$L_{2,2}(x) = \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \frac{(x - 15)(x - 18)}{(22 - 15)(22 - 18)}$$

$$p_2(x) = \sum_{k=0}^2 y_k \cdot L_{2,k}(x) = y_0 L_{2,0}(x) + y_1 L_{2,1}(x) + y_2 L_{2,2}(x)$$

$$p_2(x) = 24 \frac{(x - 18)(x - 22)}{(15 - 18)(15 - 22)} + 37 \frac{(x - 15)(x - 22)}{(18 - 15)(18 - 22)} + 25 \frac{(x - 15)(x - 18)}{(22 - 15)(22 - 18)}$$

$$p_2(16) = 24 \frac{(16 - 18)(16 - 22)}{(15 - 18)(15 - 22)} + 37 \frac{(16 - 15)(16 - 22)}{(18 - 15)(18 - 22)} + 25 \frac{(16 - 15)(16 - 18)}{(22 - 15)(22 - 18)} = 0.5$$

- c) Lineær interpolasjon: $v(t) = v(15) + \frac{f(20) - f(15)}{(20 - 15)}(t - 15) = 362.78 + 30.914(t - 15) \simeq 100.93 + 30.914t$

Oppgave 1.12.3:

- a) $f(x) = \log(x)$ Lineær interpolasjon: $f(x) = f(x_0) + (x - x_0) \cdot f[x_1, x_0] = f(4) + \frac{f(6) - f(4)}{(6 - 4)} = 0.6020600 + 0.0880046 \simeq 0.690106$
 $f(5) \simeq f(4) + (5 - 4) \cdot 0.0880046 \simeq 0.690106$

- b) $f(x) = \log(x)$

Lineær interpolasjon:

$$f(x) = f(x_0) + (x - x_0) \cdot f[x_1, x_0] = f(4) + \frac{f(5.5) - f(4.5)}{(5.5 - 4.5)} = 0.0880046$$

$$f(5) \simeq f(4.5) + (5.5 - 4.5) \cdot 0.0871502 \simeq 0.696788$$

$$f[x_1, x_0] = \frac{f(5.5) - f(4.5)}{(5.5 - 4.5)} = 0.0871502$$

- c) Kvadratisk interpolasjon: $x_0 = 4.5$, $x_1 = 5.5$, $x_2 = 6$ og dermed:

$$f[x_1, x_0] = 0.0871502 \text{ (beregnet i forrige opgavedel)}$$

$$f[x_2, x_1] = [f(6) - f(5.5)] / (6 - 5.5) = 0.0755772$$

$$f[x_2, x_1, x_0] = \frac{f[x_2, x_1] - f[x_1, x_0]}{(6 - 4.5)} = -0.0077153$$

$$f(5) = 0.696788 + (5 - 4.5)(5 - 5.5)(-0.0077153) = 0.698717$$

- d) Avvik for del a) : $e = \frac{f_{eksakt}(5) - f_{estimert}(5)}{f_{eksakt}(5)} \simeq \frac{0.69897 - 0.690106}{0.69897} \simeq 0,012658 \simeq 1,27\%$

$$\text{Avvik for del b) : } e = \frac{f_{eksakt}(5) - f_{estimert}(5)}{f_{eksakt}(5)} \simeq \frac{0.69897 - 0.696788}{0.69897} \simeq 0,00312 \simeq 0,3\%$$

$$\text{Avvik for del c) : } e = \frac{f_{eksakt}(5) - f_{estimert}(5)}{f_{eksakt}(5)} \simeq \frac{0.69897 - 0.698717}{0.69897} \simeq 0,0004 \simeq 0,04\%$$

Konklusjon:

- Estimeringsfeilen avtar når målepunktene er nærmere til det interpolerte punktet.
- Estimeringsfeilen avtar når graden av det interpolerende polynomet øker.

Oppgave 1.12.4:

- a)

$$p_n = \sum_{i=0}^{n-1} f(x_i) l_i(x)$$

$$p_3(x) = 7 \frac{(x-2)(x-3)(x-4)}{(0-2)(0-3)(0-4)} + 11 \frac{(x-0)(x-3)(x-4)}{(2-0)(2-3)(2-4)} +$$

$$28 \frac{(x-0)(x-2)(x-4)}{(3-0)(3-2)(3-4)} + 63 \frac{(x-0)(x-2)(x-3)}{(4-0)(4-2)(4-3)}$$

$$p_3(x) = -7 + 24(x-2)(x-3)(x-4) + \frac{11}{4}(x)(x-3)(x-4) -$$

$$\frac{28}{3}(x)(x-2)(x-4) + \frac{63}{8}(x)(x-2)(x-3)$$

$$p_3(x) = x^3 - 2x + 7$$

- b)

```

c) clc;
   x=[0,2,3,4];
   y=[7,11,28,63];
   Lagrange(x,y,4,2.5);
   >>yy =
       17.6250
   % Lagrange programmet - se eksempel 1.4.2 i dette notatet.

```

Oppgave 1.12.5:

a)

$$p_0 = 2$$

$$p_1 = 2 + c(x + 2)$$

$$14 = 2 + c(1), c = 12$$

$$p_2 = 2 + 12(x + 2) + c(x + 2)(x + 1)$$

$$4 = 2 + 24 + c(2), c = -11$$

$$p_3 = 2 + 12(x + 2) - 11(x + 2)(x + 1) + c(x + 2)(x + 1)x$$

$$2 = 2 + 36 - 66 + c(6), c = 5$$

$$p_4 = 2 + 12(x + 2) - 11(x + 2)(x + 1) + 5(x + 2)(x + 1)x + c(x + 2)(x + 1)(x)(x - 1)$$

$$2 = 2 + 48 - 132 + 120 + c(24), c = -\frac{3}{2}$$

Interpolasjonspolynomet er da:

$$p_4(x) = 2 + 12(x + 2) - 11(x + 2)(x + 1) + 5(x + 2)(x + 1)x - \frac{3}{2}(x + 2)(x + 1)(x)(x - 1)$$

$$p_4(x) = -\frac{3}{2}x^4 + 2x^3 + \frac{11}{2}x^2 - 8x + 4$$

b)

c)

Oppgave 1.12.6:

a)

$$\begin{array}{lcl}
 x_0 = & y_0 = f[x_0] = 2 & \\
 & \swarrow f[x_0, x_1] = 1 & \\
 x_1 = 2 & y_1 = f[x_1] = 4 & \swarrow f[x_0, x_1, x_2] = -3 \\
 & \swarrow f[x_1, x_2] = 8 & \swarrow f[x_0, x_1, x_2, x_3] = 4 \\
 x_2 = 3 & y_2 = f[x_2] = -4 & \swarrow f[x_1, x_2, x_3] = 51/3 \\
 & \swarrow f[x_2, x_3] = 43 & \\
 x_3 = 5 & y_3 = f[x_3] = 82 & \\
 f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{4 - 2}{2 - 0} = 1
 \end{array}$$

$$p_3(x) = \sum_{j=0}^3 f[x_0, x_1, \dots, x_j] \prod_{i=0}^{j-1} (x - x_i)$$

$$p_3(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$$

$$p_3(x) = 2 + 1(x) - 3(x)(x - 2) + 4(x)(x - 2)(x - 3)$$

$$p_3(x) = 4x^3 - 23x^2 + 31x + 2$$

b)

c)

Oppgave 1.12.7:

- a) MATLAB programmet utfører 3 interpolasjon på datapunktenene, lineær, cubic og spline.
- b) Spline-interpolasjon er en generalisering av lineær/polynom interpolasjon, der en bruker polynom med førstegrad/ $(n - 1)$ 'te grad mellom datapunktene. Istedenfor å bruke samme polynom til å interpolere alle verdier av x velger en isteden å la definisjonen av polynomet variere fra dataintervall til dataintervall. Polynomene for hvert delintervall konstrueres slik at at en oppnår en viss grad av glatthet i interpolanten, for alle verdier av argumentet x . Spline polynomial interpolasjon er glatttest da den forutsetter ikke bare kontinuitet og derivasjonskontinuitet i endepunktene men også at den dobbelt deriverte er kontinuerlig i ende punktene.